

ENUNCIADOS DE TRABAJOS PRÁCTICOS EVALUABLES

INGENIERÍA Y CALIDAD DE SOFTWARE

Índice

PRÁCTICO 1 - REQUERIMIENTOS ÁGILES – User Stories	3
SOLUCIÓN PRÁCTICO 1 - REQUERIMIENTOS ÁGILES – User Stories	4
PRÁCTICO 2 - REQUERIMIENTOS ÁGILES – User Stories y Estimaciones	11
SOLUCIÓN PRÁCTICO 2 - REQUERIMIENTOS ÁGILES – User Stories	13
PRÁCTICO 3 - REQUERIMIENTOS ÁGILES – User Stories, Estimaciones y MVP	19
SOLUCIÓN PRÁCTICO 3 - REQUERIMIENTOS ÁGILES – User Stories	22
MVP (Producto Mínimo Viable)	22
User stories incluidas en el MVP	22
PRÁCTICO 4 - SCM - Herramientas de SCM (Evaluable)	27
PRÁCTICO 5 - SCM - Uso del Repositorio (Evaluable)	28
PRÁCTICO 6 - TDD (Evaluable)	29
PRÁCTICO 7 - SCRUM - Ejercicio de aplicación (Evaluable)	30
PRÁCTICO 8 – SCRUM - Planificación de Release y de Sprint (Evaluable) - BONUS	33
PRÁCTICO 9 - TESTING - Métodos de Caja Negra	41
SOLUCIÓN PRÁCTICO 9 - TESTING	43
PRÁCTICO 10 - TESTING - Métodos de Caja Negra	51
SOLUCIÓN PRÁCTICO 10 - TESTING	52
PRÁCTICO 11 - TESTING – Ejecución de Casos de Prueba (Evaluable)	54
PRÁCTICO 12 - KANBAN – Ejercicio de aplicación (Evaluable)	55
PRÁCTICO 13 - SCRUM - Dinámica de Retrospectiva (Evaluable)	56
PRÁCTICO 14 - DESIGN THINKING – Publicidad de Instagram (Evaluable)	57

PRÁCTICO 1 - REQUERIMIENTOS ÁGILES – User Stories

Unidad:	Unidad Nro. 2: Gestión Lean-Ágil de Productos de Software
Consigna:	Identificar y escribir las US identificadas en el Dominio presentado para el práctico
Objetivo:	Aplicar los conceptos teóricos desarrollados en clase sobre User Stories.
Propósito:	Familiarizarse con los conceptos de requerimientos ágiles y en particular con Historias de Usuario (User Stories), Épicas y Temas
Entradas:	Conceptos teóricos sobre el tema, desarrollados en clase. Bibliografía referenciada en la modalidad académica. Enunciado, consigna y tarjeta de US.
Salida:	- Identificación de los roles principales - US identificadas con sus tarjetas completas Este práctico no se entrega y por lo tanto no tiene nota. El tema se evalúa en el parcial.
Instrucciones:	Los docentes representarán a expertos del dominio que expresarán sus necesidades vinculadas a desarrollar un software para un servicio de transporte. Los estudiantes realizarán preguntas con el objetivo de acordar juntos el alcance del proyecto, y determinar las user stories correspondientes y definir las pruebas de aceptación de usuario para cada una. Cada grupo presentará el resultado obtenido al final de la clase al resto del curso.

Aplicación Mis Gastos Familiares

Objetivo del producto: facilitar a las personas la gestión de sus gastos cotidianos y poder acceder a información relacionada a estos.

A continuación, se transcribe parte de la entrevista realizada al experto en el dominio:

Product Owner (PO): ¿Cómo debe visualizarse la planilla de gastos?

Experto en el Dominio (ED): Similar a una tabla en Excel, con columnas donde se pueda indicar el monto, el tipo de gasto, y la fecha en que se realizó el gasto.

PO: ¿La fecha del gasto es la fecha actual? ¿Se toma automáticamente?

ED: Debería mostrar la fecha actual, pero permitir modificarse.

PO: ¿Cuáles son los tipos de gastos permitidos?

ED: Cada persona debería poder registrar sus tipos de gastos, así como indicar si el gasto es propio o de otra persona (por ejemplo, de su esposa).

PO: ¿El nombre o la relación con el usuario debe indicarse?

ED: Debe indicarse el nombre y apellido. También por defecto debe mostrarse el nombre y el apellido del usuario logueado, permitiendo modificarlo, y si alguna vez se registró el nombre y apellido que se comienza a ingresar (no importa si esta vez o una vez anterior), el sistema debería mostrar aquellos nombres y apellidos que comiencen de forma similar, como hace el Excel.

PO: Ah ¿Es decir que el usuario debe registrarse y cuando va a usar la aplicación debe iniciar sesión?, y ¿la sesión caduca en algún momento?

ED: El usuario debe registrarse para permitirle descargar la aplicación, y la primera vez debe iniciar sesión, pero luego se consideran los datos de la sesión registrados, salvo que el usuario decida de loguearse. La sesión no caduca nunca.

PO: ¿Y la planilla de gastos muestra todos los gastos? ¿Cómo se ordenan?

ED: Por defecto se deben mostrar todos los gastos del mes en curso ordenados desde el gasto más actual, pero el sistema debería permitir ver cualquier período que el usuario quiera, y que pueda filtrar por tipo de gasto, por responsable de gasto, por rango de montos. Además debe poder modificar el criterio de ordenamiento. Y para cada filtro que se aplique arriba se debe mostrar el total de gastos según el filtro aplicado.

SOLUCIÓN PRÁCTICO 1 - REQUERIMIENTOS ÁGILES – User Stories

ROL IDENTIFICADO:

Rol de Usuario	Descripción
 Ejecutor de Gastos	Persona que por lo general está familiarizada con el uso de aplicaciones web mobile, y expectante de las últimas novedades en relación ello. Está familiarizado con el uso de smartphones y puede instalar la aplicación. Utilizará la aplicación con mucha frecuencia y requerirá de su simplicidad. Su expectativa es poder visualizar ordenados sus gastos para realizar análisis de los mismos en el momento que desee.

MVP (PRODUCTO MÍNIMO VIABLE) PARA EL PRODUCTO DE SOFTWARE MIS GASTOS FAMILIARES:

Objetivo:

Permitir que los Ejecutores de Gastos puedan registrar los gastos realizados y luego consultarlos para realizar un análisis.

Criterio:

Las user stories incluidas en el MVP permiten validar la idea de negocio en el mercado, las mismas se focalizan en la funcionalidad que otorga al Interesado la posibilidad de registrar gastos y de poder visualizarlos en una planilla. La funcionalidad referida al registro de los gastos de otras personas, así como también las distintas formas de ordenamiento y filtro de los gastos, no aportan valor significativo para la validación del mercado, con lo cual las mismas se dejarán para desarrollar en futuras iteraciones.

HISTORIAS DE USUARIO

(LAS USER STORIES REMARCADAS EN AMARILLO SON LAS INCLUIDAS EN EL MVP).

Frases Verbales (Phrasal Verbs) para las User Stories:

- Guardar gasto
- Generar listado de gastos
- Registrar usuario
- Loguear usuario
- Cerrar sesión de usuario
- Registrar tipo de gasto
- Registrar responsable de gastos
- Consultar gastos

HISTORIAS DE USUARIO CON SU ESTIMACIÓN

Guardar gasto

3

Como Ejecutor de Gastos quiero guardar un gasto para tener información de los gastos que realizo.

Criterios de Aceptación:

- Para cada gasto se debe ingresar: la fecha, el tipo de gasto, una descripción del gasto, el monto, indicar si es propio o no, y el nombre y apellido de la persona responsable del gasto.
- Se debe mostrar por defecto la fecha actual y permitir que se pueda modificar.
- La fecha del gasto nunca podrá ser mayor a la actual (no gastos a futuro).
- Se debe mostrar el nombre y apellido del usuario logueado como responsable del gasto y permitir modificarlo.
- Se puede seleccionar el tipo de gasto (opcional); de los definidos por el Ejecutor de Gastos.

Pruebas de Usuario

- ☐ Probar guardar un gasto con todos los datos (fecha del gasto, el tipo de gasto, la descripción del gasto, el monto, gasto propio o no, nombre y apellido del responsable. (pasa)
- ☐ Probar guardar un gasto modificando la fecha actual y la misma es menor a la actual (pasa).
- ☐ Probar guardar un gasto modificando la fecha actual y la misma es mayor a la actual (falla).
- ☐ Probar guardar un gasto modificando el nombre del responsable de gasto (pasa).
- ☐ Probar guardar un gasto sin ingresar el nombre del responsable del gasto (falla).
- ☐ Probar guardar un gasto sin guardar el tipo de gasto (pasa).
- ☐ Probar guardar un gasto sin guardar la descripción del gasto (falla).

Justificación de Estimación:

- **Complejidad:** Media. Los datos a ingresar son pocos y también son datos simples.
- **Esfuerzo:** Medio, se requiere esfuerzo de validación de los datos ingresados.
- **Incertidumbre:** No hay incertidumbre debido a que el requerimiento está claro.

Generar listado de gastos

3

Como Ejecutor de Gastos quiero visualizar mis gastos para informarme de mi situación financiera en un período de tiempo.

Criterios de Aceptación:

- Se debe visualizar los gastos en un formato de matriz, con columnas para la fecha, el tipo de gasto, una descripción del gasto, el monto del gasto, si es propio o no, y el nombre y apellido de la persona responsable del mismo.

Pruebas de Usuario

- ☐ Probar generar un listado de gastos registrados para un período correcto. (pasa)
- ☐ Probar generar un listado de gastos registrados para un período correcto, que no tenga gastos en ese periodo (pasa)
- ☐ Probar generar un listado de gastos registrados para un período incorrecto (fechas a futuro) (falla)
- ☐ Probar generar un listado de gastos registrados para un período invalido (falla)

Justificación de Estimación:

- **Complejidad:** user story de complejidad media, ya que, si bien se van a utilizar los componentes que están provistos por el framework mobile, será necesaria la evaluación de los mismos para la generación y visualización de la planilla y para el ordenamiento por cada columna de la grilla.
- **Esfuerzo:** medio, relacionado a la diagramación y visualización de la planilla.
- **Incertidumbre:** Baja. Está claro el requerimiento y no hay duda técnica, salvo respecto a la selección de la mejor opción para lograr una buena visualización del contenido de la planilla.

Registrar Usuario [user story canónica]	1
Como Ejecutor de Gastos quiero registrarme en la aplicación para comenzar a analizar mis gastos. Criterios de Aceptación: • Debe ingresar: dni, nombre, apellido, usuario y contraseña (todos requeridos) • Características de la contraseña: debe tener al menos 8 caracteres, con algún carácter especial. • No debe existir otro usuario con el mismo DNI o nombre de usuario. Pruebas de Usuario ☐ Probar registrar un Ejecutor de Gastos con un DNI no registrado. (pasa) ☐ Probar registrar un Ejecutor de Gastos con un DNI existente. (falla) ☐ Probar registrar un Ejecutor de Gastos con algún dato requerido faltante (falla) ☐ Probar registrar un Ejecutor de Gastos con una contraseña de menos de 8 caracteres (falla) ☐ Probar registrar un Ejecutor de Gastos con una contraseña de 8 caracteres sin ningún carácter especial (falla) ☐ Probar registrar un Ejecutor de Gastos con una contraseña de 8 caracteres sin ningún carácter especial (falla)	

Canónica: la historia de usuario no tiene incertidumbre, además la cantidad de datos a registrar y a validar son pocos.

Justificación de Estimación:

- **Esfuerzo:** Bajo, la cantidad de datos a solicitar y validar es baja.
- **Complejidad:** Baja, en la mayoría de los framework modernos es un contenido provisto.
- **Incertidumbre:** Nula, no hay incertidumbre respecto al requerimiento, la necesidad está clara.

Loguear usuario

1

Como Ejecutor de Gastos quiero iniciar sesión en la aplicación para poder consultar mis gastos o registrar gastos nuevos.

Criterios de Aceptación:

- Debe loguearse con nombre de usuario y contraseña.

Pruebas de Usuario

- ☐ Probar loguear un Ejecutor de Gastos con un usuario y contraseña existente. (pasa)
- ☐ Probar loguear un Ejecutor de Gastos con un usuario y contraseña inexistente. (falla)

Justificación de Estimación:

- **Complejidad:** user story simple ya que requiere un ingreso de datos mínimos.
- **Esfuerzo:** Se requiere esfuerzo mínimo de validación.
- **Incertidumbre:** No hay incertidumbre debido a que el requerimiento está claro.

Cerrar sesión de usuario

1

Como Ejecutor de Gastos quiero cerrar la sesión para mantener la seguridad de mi información luego de usar la aplicación.

Pruebas de Usuario

- ☐ Probar cerrar la sesión de un usuario previamente logueado. (pasa)
- ☐ Probar cerrar la sesión de un usuario no logueado previamente. (falla)

Justificación de Estimación:

- **Complejidad:** user story simple ya que no requiere de ingreso de datos ni validaciones importantes.
- **Esfuerzo:** Se requiere esfuerzo mínimo de validación.
- **Incertidumbre:** No hay incertidumbre debido a que el requerimiento está claro.

Registrar un tipo de gasto

1

Como **Ejecutor de Gastos** quiero registrar un tipo de gastos para clasificar los gastos al momento de registrarlos.

Criterios de Aceptación:

- Debe ingresar para cada tipo de gasto: nombre del tipo de gasto y comentario (opcional).

Pruebas de Usuario

- ☐ Probar registrar un tipo de gasto con un nombre inexistente. (pasa)
- ☐ Probar registrar un tipo de gasto con un nombre existente. (falla)
- ☐ Probar registrar un tipo de gasto con todos los datos (pasa)
- ☐ Probar registrar un tipo de gasto con el nombre, pero sin el comentario del tipo de gasto (pasa)

Justificación de Estimación:

- **Complejidad:** user story simple ya que requiere un ingreso de datos mínimos y validaciones simples.
- **Esfuerzo:** Se requiere esfuerzo mínimo vinculado a validaciones simples de unicidad.
- **Incertidumbre:** Nula, no hay duda respecto a los datos y validaciones requeridos para registrar un tipo de gasto.

Registrar responsable de gastos

1

Como Ejecutor de Gastos quiero registrar un responsable de gastos para asignar gastos a otras personas, además de a mí.

Criterios de Aceptación

- Se debe ingresar para cada responsable de gasto son: nombre y apellido (requeridos)

Pruebas de Usuario

- ☐ Probar registrar un responsable de gastos existente (nombre y apellido registrados). (falla)
- ☐ Probar registrar un responsable de gastos inexistente (nombre y apellido no registrados). (pasa)

Justificación de Estimación:

- **Complejidad:** user story simple ya que requiere un ingreso de datos mínimos y validaciones simples.
- **Esfuerzo:** Se requiere esfuerzo mínimo vinculado a validaciones simples de unicidad.
- **Incertidumbre:** Nula, no hay duda respecto a los datos y validaciones requeridos para registrar un responsable de gastos.

Consultar gastos

3

Como **Ejecutor de Gastos** quiero consultar mis gastos de la planilla de gastos y ordenarlos para facilitar la visualización.

Criterios de aceptación:

- Los filtros permitidos son: período, tipo de gasto, responsable de gasto, rango de montos.
- Se puede ordenar por tipo de gasto, responsable, monto o fecha de forma ascendente o descendente.
- Para cada filtro que se aplique, en la parte superior de la planilla se debe mostrar el total de gastos según el filtro aplicado.

Pruebas de Usuario

- ☐ Probar filtrar los gastos por un tipo de gasto. (pasa)
- ☐ Probar filtrar los gastos por un tipo de gasto, responsable, fecha y monto. (pasa)
- ☐ Probar ordenar los gastos por responsable (pasa)
- ☐ Probar ordenar los gastos por descripción. (falla)
- ☐ Probar ordenar los gastos por fecha ascendente (pasa)
- ☐ Probar ordenar los gastos por monto de gasto descendente (pasa)

Justificación de Estimación:

- **Complejidad:** Baja, la misma está asociada al desarrollo de los componentes para filtrar, ordenar los gastos y realizar los cálculos.
- **Esfuerzo:** medio, relacionado al logro de una buena performance en la ejecución de los filtros y ordenamientos de los gastos de la planilla y a las pruebas necesarias para validar filtros y cálculos de datos sumariadas.
- **Incertidumbre:** Baja. Está claro el requerimiento y no hay duda técnica.

PRÁCTICO 2 - REQUERIMIENTOS ÁGILES – User Stories y Estimaciones

Unidad:	Unidad Nro. 2: Gestión Lean-Ágil de Productos de Software
Consigna:	Identificar y escribir las US identificadas en el Dominio presentado para el práctico
Objetivo:	Aplicar los conceptos teóricos desarrollados en clase sobre User Stories.
Propósito:	Familiarizarse con los conceptos de requerimientos ágiles y en particular con US, Épicas, Temas y MVP..
Entradas:	Conceptos teóricos sobre el tema, desarrollados en clase. Bibliografía referenciada sobre el tema. Enunciado, consigna y tarjeta de US.
Salida:	- Identificación de los roles principales - El MVP explicando el alcance propuesto y justificando la inclusión de las User Stories seleccionadas y el porqué de aquellas excluidas en el mismo - US identificadas con sus tarjetas completas - User Story canónica - User Stories estimadas Este práctico no se entrega y por lo tanto no tiene nota. El tema se evalúa en el parcial.
Instrucciones:	Los docentes representarán a expertos del dominio que expresarán sus necesidades vinculadas al desarrollar un software para el servicio de EcoHarmony Park. Los estudiantes realizarán preguntas con el objetivo de acordar juntos el alcance del proyecto, determinar las User Stories correspondientes y definir las pruebas de aceptación de usuario para cada una. Cada grupo presentará el resultado obtenido al final de la clase al resto del curso.

EcoHarmony Park: Descripción del dominio

Los dueños del bioparque “EcoHarmony Park” están evaluando el desarrollo de una aplicación móvil (Android, iOS) ya que la cantidad de visitantes anuales aumentaron significativamente al igual que los avances tecnológicos. La misma tiene como objetivo mejorar la experiencia de los visitantes al proporcionar información útil y enriquecedora sobre las exhibiciones, los horarios de alimentación de los animales, los senderos para caminar y otros aspectos del parque como la autogestión al momento de realizar la compra de entradas.

Hemos tenido el privilegio de entablar una conversación con Miguel Rodríguez, uno de los propietarios de EcoHarmony Park, y a continuación, compartimos los detalles de la interacción:

El Doce: ¡Hola! Estoy aquí en el hermoso bioparque "EcoHarmony Park" con el Sr. Rodríguez, para hablar sobre una emocionante novedad: ¡una nueva aplicación móvil que mejorará la experiencia de los visitantes aquí en el parque! ¿Me podrías contar brevemente en qué consiste la aplicación?

Miguel: ¡Hola! Gracias por tomarte el tiempo para hablar conmigo. Estoy encantado de compartir más detalles sobre la idea de nuestra aplicación. En primer lugar, ofrecerá a los visitantes mapas interactivos que les ayudarán a explorar el parque de manera más eficiente. Asimismo, proporcionará información detallada sobre cada exhibición y horario de alimentación de cada especie que cuidamos. También se podrá autogestionar la compra de entradas al parque, pudiendo realizar directamente el pago a través de la pasarela de Mercado Pago, lo que permitirá agilizar el ingreso. Por último, los visitantes pueden inscribirse a las diferentes actividades diarias que posee el parque.

El Doce: Wow! Que interesante ¿Podrías explicarme cómo puede un visitante acceder a los horarios de alimentación a través de la aplicación?

Miguel: ¡Claro! Una vez que descarguen la aplicación y la abran, encontrarán una sección dedicada a "Horarios de Alimentación disponibles". Cada horario estará detallado de manera clara, indicando la especie, nombre del animal, edad, hora programada, sector del parque (terrestres, acuáticos o aéreos) y cuidador encargado (nombre y apellido). Por otra parte, aquellos que estén próximos a comenzar (faltando una hora) los mostraremos con un mensaje que diga: ¡Apúrate que ya comienza!

Es importante considerar que los horarios se mostrarán en orden, comenzando con los más próximos a la fecha actual y extendiéndose hasta una semana en el futuro. Para que sea más amigable la interfaz pensamos en proporcionar la posibilidad de filtrar por la fecha en la que el visitante estará en el parque y que se visualicen únicamente los horarios correspondientes a la misma.

Además, en una versión posterior, los visitantes podrán configurar notificaciones para recibir recordatorios de los horarios de alimentación que deseen seguir de cerca.

El Doce: ¡Eso suena muy útil! ¡Al igual que el mapa interactivo! ¿Podrías comentarme cuáles serán los elementos y características del parque que estarán disponibles para visualizar en él? ¿Qué acciones podrán llevar a cabo los visitantes a través de esta herramienta?

Miguel: Este mapa ofrece una vista panorámica de todo el parque, dividido en sectores diferenciados por colores. Asimismo, el mapa contiene íconos informativos: “Inodoro” para identificar los baños distribuidos en el parque, una “Bolsa” para indicar los puntos de venta, y un “Micrófono” para el show especial del día. Los visitantes pueden hacer click en este último ícono para obtener el nombre del show y su horario, en caso de que haya un show programado. En una futura mejora pensamos incluir en este mapa la ubicación del visitante en tiempo real.

El Doce: Para alguien como yo, con un sentido de la orientación no muy desarrollado, suena como una herramienta imprescindible, ¡jaja! ¿Hay otros aspectos del parque que también planean ofrecer a través de la aplicación? ¿Habías mencionado la inscripción a las actividades, no es cierto?

Miguel: Sí! me estaba olvidando de comentarlo. El parque ofrece la posibilidad de realizar actividades de una hora sumamente enriquecedoras para los visitantes, entre las que se encuentran “Tirolesa”, “Safari”, “Palestra” y “Jardinería”. Para poder realizar la inscripción deben seleccionar del conjunto de actividades la que desean realizar, eligiendo en primer lugar el horario y completando luego los datos del visitante: nombre, DNI, edad y talla de vestimenta si la actividad lo demanda. Finalmente, para concluir el proceso de inscripción, pedimos a los visitantes que acepten los términos y condiciones específicos de la actividad en la que participarán. Nos interesa gestionar la inscripción de las actividades porque tienen cupos limitados y es muy útil para planificarlas con anterioridad según los inscriptos que tengamos.

El Doce: Para finalizar esta pequeña entrevista, ¿Podrías explicarme el proceso de compra de entradas a través de la app?

Miguel: ¡Pero claro! Para realizar la compra de entradas el visitante debe registrarse en la aplicación, ingresando un mail y una contraseña. A continuación, debe ingresar a la sección “Comprar entradas” y allí indicar la fecha de visita deseada, la cantidad de entradas requeridas (que no puede superar las 10) y la edad de cada visitante. Finalmente, el sistema mostrará el monto total y se debe seleccionar la forma de pago: efectivo en caso de querer pagar en boletería o con tarjeta, donde se redirigirá al usuario a la página de Mercado Pago para completar el proceso de forma segura. Finalmente, recibirán un mensaje de confirmación vía mail y sus entradas serán verificadas al momento de ingresar al parque. La posibilidad de comprar las entradas mediante la aplicación nos parece fundamental para comenzar a probar la recepción de la aplicación, incluyendo el pago electrónico ya que creemos que será el más usado.

El Doce: ¡Muy claro! Parece que la aplicación requiere mucha atención. ¿Tienen previsto contratar personal adicional para mantener actualizados los horarios, las actividades y todo lo que implica?

Miguel: No, no hay problema en ese aspecto. Planeamos gestionar esa tarea por fuera de la aplicación, a través de nuestro equipo de administradores, que se encargará de mantener todo actualizado.

El Doce: Ah, entiendo. ¡Gracias por compartir todos estos emocionantes detalles! Más adelante coordinamos otra nota para ver esta aplicación en funcionamiento.

SOLUCIÓN PRÁCTICO 2 - REQUERIMIENTOS ÁGILES – User Stories

Descripción: Desarrollar una app mobile que ofrezca a los visitantes la posibilidad de inscribirse en diversas actividades, acceder al mapa del parque y consultar los horarios de las exhibiciones, todo sin requerir un registro inicial. Además, la aplicación permitirá a los usuarios comprar entradas para el parque, momento en el cual se les solicitará registrarse

Justificación: se quiere probar si habrá recepción para utilizar la app por parte del visitante. Permitiendo las funcionalidades básicas para una visita satisfactoria. La gestión por parte de los administradores como el registro de las actividades disponibles, las exposiciones y la configuración del show del día se podrá realizar por base de datos, sin necesidad de incluir funcionalidades para el administrador.

Fuera de Alcance:

- Recordatorios: marcar recordatorio y enviar notificaciones de estos
- Registro de los sectores y las actividades.
- Visualizar ubicación propia en el mapa interactivo
- Administración de los horarios del show del día, y los horarios de alimentación

Listado de Users MVP:

Visitante:

- Inscribir en actividad del parque
- Ver mapa del parque
- Ver horarios de alimentación
- Comprar entradas (incluyendo derivación a MP y envío de mail, que pueden ser US separadas)
- Registrar usuario

Ver horarios de alimentación COMO visitante QUIERO ver los horarios de alimentación PARA planificar el recorrido dentro del bioparque	3
Criterios de Aceptación: • Para cada horario de alimentación se deben visualizar especie, nombre del animal, edad, hora programada, sector del parque (terrestres, acuáticos o aéreos) y cuidador encargado (nombre y apellido). • Se deben mostrar primero los horarios más próximos a la fecha actual. • No deben mostrarse horarios de alimentación con fechas pasadas. • Se deben mostrar horarios de alimentación planificados hasta una semana en el futuro. • Se debe permitir filtrar por la fecha en la que el visitante estará en el parque teniendo en cuenta que debe ser dentro de la semana próxima a la fecha actual. • Se debe visualizar el mensaje ¡Apúrate que ya comienza! para aquellos que en una hora comienzan.	
Pruebas de usuario: - Probar visualizar los horarios de alimentación ordenados de manera que los más próximos a la fecha actual se muestran primero, hasta una semana posterior; y por cada horario se visualiza la especie, el nombre del animal, la edad, hora programada, sector del parque y el cuidador encargado (pasa) - Probar visualizar los horarios filtrando por una fecha que se encuentra posterior a una semana (falla) - Probar visualizar los horarios filtrando por una fecha pasada (falla) - Probar visualizar los horarios filtrando por una fecha inferior a una semana (pasa) - Probar visualizar el mensaje “Apúrate que ya comienza” para los horarios en los que falta una hora o menos (pasa) - Probar visualizar los horarios de alimentación cuando no haya ninguno cargado (pasa)	

Justificación de Estimación:

- **Complejidad:** Media, existen varias validaciones a realizar en cuanto a la visualización de los horarios debido a que no se deben mostrar horarios de fechas pasadas ni de fechas posteriores a una semana. A su vez, se requiere un ordenamiento de los mismos.
- **Esfuerzo:** Medio, se requiere un diseño previo para confeccionar una interfaz que muestre los horarios de manera clara e intuitiva y además la consulta a la base de datos que debemos confeccionar requiere muchos datos.
- **Incertidumbre:** Baja, no hay incertidumbre técnica pero sí algunas cuestiones de visualización de los horarios

Ver mapa del parque COMO visitante QUIERO ver mapa del parque PARA obtener información sobre la distribución geográfica del parque	8
Criterios de Aceptación: • Debe mostrarse el mapa dividido en sectores diferenciados por colores. • El mapa debe contener los siguientes iconos: “Inodoro”, “Bolsa”, “Micrófono” • El icono “Inodoro” debe estar posicionado en donde están los baños • El icono “Bolsa” debe estar posicionado donde hay tiendas • El icono “Micrófono” debe estar posicionado en donde es el show del día • Los sectores contenidos en el mapa deben ser: “terrestres”, “acuáticos” y “aéreos” • Los visitantes pueden hacer click en icono del “Micrófono” para obtener el nombre del show y su horario.	
Pruebas de usuario: - Probar ver el mapa del parque con los sectores “terrestres”, “acuáticos” y “aéreos” diferenciados por colores, visualizando los iconos “Inodoro”, “Bolsa”, “Micrófono” en sus correspondientes lugares - Probar visualizar la información del show (nombre del show y horario) al seleccionar el “Micrófono” un día que hay show - Probar visualizar la información del show al presionar el “Micrófono” un día en el que no se realizará ningún show (falla)	

Justificación de Estimación:

- **Complejidad:** Medio, ya que requiere que el mismo sea interactivo para el usuario (se pueden presionar elementos para obtener más información).
- **Esfuerzo:** Hay un alto esfuerzo en el diseño del mapa interactivo respetando su geografía y posicionando los iconos en el lugar correcto y los colores de los sectores.
- **Incertidumbre:** Media, no hay incertidumbre de negocio pero sí sobre cómo desarrollar la vista del mapa

User canónica:

Registrar show del día COMO administrador QUIERO registrar el show del día PARA más visitantes asistan	1
Justificación: sirve como metro patrón para comparar con el resto de las historias, debido a su baja complejidad y esfuerzo y a que no tiene incertidumbre técnica, ni de negocio. Estimación: Esfuerzo: Bajo, requiere un diseño sencillo de interfaz al tratarse de un simple formulario que debe contener dos campos. Complejidad: Baja, se deben ingresar pocos datos (nombre del show y horario) y no se deben realizar demasiadas validaciones. La lógica detrás de la implementación es simple. Incertidumbre: Baja, se tiene una comprensión sólida tanto de los requisitos técnicos como de negocio. La complejidad respecto a cómo mostrar el ícono asociado está en otra US.	

Otras US

Comprar entradas COMO visitante QUIERO comprar una entrada PARA asegurar mi visita al parque	8
Criterios de Aceptación: • Debe indicar la fecha de visita deseada, la cantidad de entradas requeridas, la edad de cada visitante y tipo de pase (VIP o regular). • La fecha de visita guiada puede ser del día actual o futuro, y debe ser en los días en que el parque está abierto. • Debe enviar un mensaje de confirmación vía mail • Debe redirigir a mercado pago al confirmar la compra si el pago es con tarjeta de credito • La fecha de la visita debe estar dentro de los días en que el parque está abierto. • Debe seleccionar la forma de pago: efectivo en caso de +querer pagar en boletería o con tarjeta • La cantidad de entradas requeridas no debe ser mayor a 10 • Al finalizar la compra se debe informar la cantidad de entradas compradas y la fecha.	
Pruebas de usuario: - Probar comprar una entrada indicando la fecha de visita dentro de los días disponibles, una cantidad de entradas menor a 10, la edad de todos los visitantes, el tipo de pase, la forma de pago con tarjeta mediante Mercado pago y recepción del mail de confirmación (pasa) - Probar comprar entradas sin seleccionar forma de pago (falla) - Probar comprar entradas ingresando una fecha de visita en la cual el parque se encuentra cerrado (falla) - Probar comprar entradas ingresando una cantidad de entradas mayor a 10 (falla)	

Justificación de Estimación:

- **Complejidad:** Media, requiere validaciones de de horarios, y una gran cantidad de datos a ingresar.
- **Esfuerzo:** Media, se requiere cierto esfuerzo para realizar la conexión para enviar los mails y para conectar la pasarela de Mercado Pago.
- **Incertidumbre:** Media, no hay incertidumbre de negocio pero sí existe desconocimiento sobre la conexión necesaria para realizar el envío de mails y con Mercado pago.

Refinamiento de la historia:

- El mensaje de confirmación tiene cantidad de entradas y fecha.
-

Inscribirme a actividad

COMO visitante QUIERO inscribirme a una actividad PARA reservar mi lugar en la misma.

3

Criterios de Aceptación:

- Debe requerir seleccionar una actividad del conjunto de actividades de la lista de “Tirolesa”, “Safari”, “Palestra” y “Jardinería”, siempre y cuando tengan cupos disponibles para el horario seleccionado
- Debe requerir seleccionar el horario dentro de los disponibles
- Debe requerir ingresar los datos del visitante: nombre, DNI, edad y talla de vestimenta si la actividad lo demanda
- Debe requerir aceptar los términos y condiciones específicos de la actividad en la que participarán.

Pruebas de usuario:

- Probar inscribirse a una actividad del listado que poseen cupos disponibles, seleccionando un horario, ingresando los datos del visitante (nombre, DNI, edad, talla de la vestimenta si la actividad lo requiere) y aceptando los términos y condiciones (pasa)
- Probar inscribirse a una actividad que no tiene cupo para el horario seleccionado (falla)
- Probar inscribirse a una actividad sin ingresar talla de vestimenta porque la actividad no lo requiere (pasa)
- Probar inscribirse a una actividad seleccionando un horario en el cual el parque está cerrado o la actividad no está disponible (falla)
- Probar inscribirse a una actividad sin aceptar los términos y condiciones de la actividad (falla)
- Probar inscribirse a una actividad sin ingresar el talla de la vestimenta requerido por la actividad (falla)

Justificación de Estimación:

- **Complejidad:** Media, requiere validaciones de horario, de cupos, de vestimenta y una gran cantidad de datos a ingresar.
- **Esfuerzo:** Medio, se requiere cierto esfuerzo para desarrollar el front del formulario de inscripción.
- **Incertidumbre:** Baja, no hay incertidumbre técnica ni de negocio.

PRÁCTICO 3 - REQUERIMIENTOS ÁGILES – User Stories, Estimaciones y MVP

Unidad:	Unidad Nro. 2: Gestión Lean-Ágil de Productos de Software
Consigna:	Identificar y escribir las US identificadas en el Dominio presentado para el práctico
Objetivo:	Aplicar los conceptos teóricos desarrollados en clase sobre User Stories.
Propósito:	Familiarizarse con los conceptos de requerimientos ágiles y en particular con US, Épicas, Temas y MVP..
Entradas:	Conceptos teóricos sobre el tema, desarrollados en clase. Bibliografía referenciada sobre el tema. Enunciado, consigna y tarjeta de US.
Salida:	- Identificación de los roles principales - El MVP explicando el alcance propuesto y justificando la inclusión de las User Stories seleccionadas y el porqué de aquellas excluidas en el mismo - US identificadas con sus tarjetas completas - User Story canónica - User Stories estimadas Este práctico no se entrega y por lo tanto no tiene nota. El tema se evalúa en el parcial.
Instrucciones:	Los docentes representarán a expertos del dominio que expresarán sus necesidades vinculadas a desarrollar un software. Los estudiantes realizarán preguntas con el objetivo de acordar juntos el alcance del proyecto, y determinar las user stories correspondientes y definir las pruebas de aceptación de usuario para cada una. Cada grupo presentará el resultado obtenido al final de la clase al resto del curso.

Recircula tus prendas - Cuida el planeta

Glosario:

- Marketplaces:** es un sitio de ventas por internet que conecta compradores y vendedores.
- Segunda selección:** se denomina así a las prendas que tienen alguna falla, mancha u otro defecto menor.
- Pasarela de pago:** es un servicio online para el cobro de transacciones con tarjetas de débito o crédito.

Descripción del Dominio

Un grupo de emprendedores preocupados por el alto impacto ambiental provocado por la industria textil quiere desarrollar una plataforma de moda circular y sustentable para comprar y revender prendas únicas. Los emprendedores apuestan a que existe un mercado para compradores de ropa usada, pero no saben si hay personas dispuestas a vender su ropa. A continuación, se transcribe parte de la entrevista realizada al experto en el dominio:

PO (Product Owner): Nos cuentan ¿cómo espera que funcione su idea?

ED (Experto del dominio): Bueno. Nuestra idea es que las personas puedan vender aquellas prendas que ya no usan y que están como nuevas o -incluso- aún con la etiqueta puesta. La plataforma funcionará como un Marketplaces en la cual los clientes podrán comprar prendas, aplicando filtros de búsqueda por distintas categorías (remeras, pantalones, camperas, etc.), marcas y estados (como nueva, nueva con etiqueta, segunda selección o segunda selección con etiqueta). Por otro lado, los vendedores eligen las prendas que desean vender y nos las envían a nosotros para que nos ocupemos de su comercialización. Nuestro proceso arranca desde la selección (donde elegimos la ropa a publicar bajo estrictos controles o la descartamos), fotografiado, publicación para su venta en nuestro ecommerce y, una vez que la prenda se vende, nosotros nos encargamos de la cobranza, de entregarla al cliente y, finalmente, del pago de la ganancia al vendedor. Nuestro modelo de negocio consiste en cobrar un % de comisión al vendedor según el precio de venta, por ejemplo: si el precio de venta es entre 1-\$1.000 cobramos un 50%, si es entre \$1.001 a \$5.000 cobramos un 45%, etc.

PO: ¡Buenísimo! Ahora, nos cuentan ¿cómo hago para vender mis prendas? ¿puedo vender cualquier cosa?

ED: Ok. Si, podés vender muchas categorías de prendas, que podrás enterarte en nuestra web. El vendedor debe llenar un formulario indicando la cantidad de prendas de cada categoría que nos van a enviar para seleccionar. Ah! pero para poder

completarlo, primero debe registrarse en nuestro sitio web con su email y contraseña o con su cuenta de Google, y debe completar sus datos (nombre completo, DNI, y teléfono). Es importante que nos aseguremos de que el teléfono sea real para poder contactarlo. Luego, deberíamos pedirle que indique su domicilio completo (por si tenemos que retirar sus prendas o entregarle su compra) y que nos indiquen un CBU o alias para realizar la transferencia bancaria cuando se venda su prenda. Volviendo a cómo vender, nos tienen que indicar la cantidad de cada categoría de prenda, una descripción de cada prenda a enviar (por ejemplo buzo rojo talla S) y opcionalmente una foto (de hasta 512kb), y, finalmente, la forma de envío: retirar en domicilio o llevarlo a un punto de recolección. Los puntos de recolección son distintas sucursales nuestras o comercios amigos que recolectan las bolsas de prendas. Tendremos muchos puntos en distintas ciudades del país, por ahora sólo 5 en Córdoba. Una vez elegida la forma de envío, deben confirmar el formulario de prendas a vender, y se enviará un mail al vendedor y a nosotros a modo de comprobante con el detalle de prendas. Es importante que sepan que sólo pueden enviar un mínimo de 10 prendas por cada vez que quieran vender.

PO: Clarísimo. ¿Cómo seleccionan las prendas? y ¿Cómo continúa el proceso?

ED: Primero, el Encargado de Logística se encargará de coordinar el retiro y el arribo de las bolsas con prendas para vender en nuestro Centro de Procesamiento, que es uno sólo aquí en Córdoba. Ahora sí, tenemos tres Analistas de Selección que se ocupan de revisar prenda por prenda para verificar que cumplen nuestras políticas de productos a vender. Ellos completarán la primera parte de una ficha básica del producto indicando categoría, marca, si está seleccionada o no. En caso de estar seleccionada, le debe asignar el estado (como nueva, nueva con etiqueta, segunda selección o segunda selección con etiqueta). Si es segunda selección, deben indicar un motivo para que el comprador se entere cuando mire el producto en el ecommerce, por ejemplo: “Tiene una pequeña mancha”. Independientemente, de si está seleccionada o no, a las prendas se les pega una etiqueta autoadhesiva con un código QR que tiene un código de producto único y permite identificar a la prenda. Las prendas que no queden seleccionadas son separadas para luego informarle el motivo de no selección y preguntarle al vendedor que desea hacer: retirar o donarla.

Una vez seleccionadas las prendas, son procesadas por 4 Analistas de Publicación que identifican cada prenda escaneando su código QR con un lector de QR, visualizan los datos cargados por el Analista de Selección, se ocupan de tomar las fotos (máximo 5 fotos, y cada una con un peso hasta 1Mb) y de completar el resto de la ficha del producto (nombre de la prenda, peso en kilogramos, precio sugerido y atributos dinámicos que describen características de la prenda según la categoría, por ejemplo: tipo de tela, color, talla, etc.) para ser publicados en el ecommerce. El precio sugerido se calcula en base a una lista de precios por categoría, marca, estacionalidad y estado de la prenda, e igualmente podrá editarlo.

Cuando la ficha de todas las prendas recibidas de un vendedor es completada, se genera la propuesta de venta y se notifica al vendedor por email que ya la tiene disponible en la plataforma. La propuesta detalla la fecha y cantidad de prendas, el listado de todas las prendas seleccionadas con su correspondiente nombre, categoría, estado, precio sugerido y la ganancia que obtendrá al venderse; y también el listado de todas las prendas que no quedaron seleccionadas. El vendedor debe ingresar a la plataforma y confirmar si desea vender todas las prendas seleccionadas, o no vender algunas o todas y, también, definir qué hacer con las prendas que quedaron no seleccionadas (si lo retira o dona). Cuando confirma, quedan las prendas seleccionadas listas para publicarse en el ecommerce, si hay prendas no seleccionadas y/o que decidió no vender, se le envía un email informando la donación o los pasos a seguir para retirarlas.

PO: ...y ahora: ¿cómo es el proceso de venta?

ED: Bueno, acá es muy parecido a cualquier ecommerce, el cliente accede a nuestra web, mira el catálogo de productos (con su nombre, precio y ficha de producto), realiza búsquedas por nombre o filtrar por categoría, estado de la prenda, marca y talla. Una vez elegida la prenda, debe registrarse o iniciar sesión para continuar con la compra, debe elegir la forma de entrega. Si elige entrega a domicilio, se calculará un costo de envío a través de una integración con un Servicio de Entregas Online que terceriza el servicio de logística. Si elige retiro en puntos de entrega, se listarán los domicilios de nuestras sucursales/comercios amigos para que elija uno y no tendrá costo. Finalmente, debe indicar el medio de pago (tarjetas de crédito o débito) para lo

cual se utilizará como pasarela de pago MercadoPago. La comercialización online de las prendas y el envío a los clientes lo vamos a realizar con una plataforma de ecommerce llamada Tiendanube hasta que podamos validar el modelo de negocio y realizar un desarrollo personalizado. Al vendedor, se le envía un email notificando de la venta con el detalle y foto de la prenda vendida, el precio de venta y la ganancia obtenida.

PO: Entonces, para cerrar el proceso, sólo falta pagarle al vendedor: ¿Cómo se realiza?

ED: Claro. Es simple, a principio de mes, el Administrador realiza las liquidaciones y pago a cada vendedor por las ventas del mes anterior que no tuvieron devolución pasado los 14 días de ocurrida la venta. El administrativo consulta todas las prendas vendidas del mes y calcula la ganancia, descontando nuestra comisión, y realiza el pago al vendedor por transferencia bancaria, registrando la misma y enviando un email con el total pagado al vendedor y el detalle de las prendas liquidadas con su ganancia.

Todo el proceso de devolución de las prendas vendidas, del envío de las prendas no seleccionadas (a donar o retirar) y del pago a los vendedores lo realizaremos inicialmente a mano.

SOLUCIÓN PRÁCTICO 3 - REQUERIMIENTOS ÁGILES – User Stories

ROLES:

- Cliente (C)
- Vendedor (V)
- Analista de Selección (AS)
- Analista de Publicación (AP)
- Encargado de Logística (EL)
- Administrador (A)

MVP (PRODUCTO MÍNIMO VIABLE)

Hipótesis: Hay vendedores dispuestos a poner en venta sus prendas que no utilizan a través de una empresa que se ocupa de todo el proceso de recepción, selección y comercialización de productos.

Descripción: Desarrollar una app web que permita registrar el envío de prendas para ser seleccionada por la empresa, recepcionar, seleccionar y digitalizar las prendas a vender y generar una propuesta de venta al vendedor.

USER STORIES INCLUIDAS EN EL MVP

Vendedor

- Crear cuenta de vendedor con email
- Enviar prendas para vender
- Confirmar propuesta de venta
- Recibir mail de prenda vendida

Analista de Selección

- Seleccionar prenda para publicar

Analista de Publicación

- Completar ficha de prenda a publicar

Otros opcionales:

- Iniciar sesión vendedor (V)
- Recibir mail de propuesta a confirmar

Criterio: Se consideraron los módulos principales asociados a los procesos para vender prendas, seleccionar y digitalizar la ficha de publicación de las prendas y confirmar la propuesta de venta por el Vendedor. No se contempla el proceso de venta y la logística hasta que se valide la viabilidad y el modelo del negocio; se resolverá inicialmente contratando una plataforma de ecommerce que también resuelve la logística.

No se incluye:

- No se incluyen inicialmente el envío de mail con pasos para retirar prendas no seleccionadas y el mail con prendas a donar que se puede resolver mediante whatsapp o manualmente.
- No se incluye funcionalidades relacionadas al ecommerce y logística de las prendas vendidas.
- No se contempla los reportes ni funcionalidades de liquidación del Administrador.
- No se contempla la creación de cuenta con Google

- No se contemplan otras funcionalidades para gestionar categorías, atributos, marcas, puntos de entrega, etc.

User Story Canónica

Crear Cuenta de Vendedor por email (alternativa: Crear Marca o Categoría de prenda)

Tamaño: 1

Justificación: No tiene incertidumbre técnica ni de negocio y es una historia de baja complejidad, por lo que sirve como metro patrón para comparar con el resto de las historias. Se puede establecer el esfuerzo requerido para el desarrollo de la historia y explicar cada tarea a realizar. NOTA: La validación del teléfono existente no es parte de esta US.

Enviar prendas para vender

8

Como Vendedor quiero enviar prendas que ya no uso para poner a la venta

Criterios de Aceptación:

- Debe ingresar la cantidad de prendas por cada categoría y una descripción de cada producto enviado.
- Puede ingresar una foto de la prenda. Cada foto no debe superar 512Kb.
- Debe indicar la forma de envío (retirar en domicilio o llevar a punto de recolección). En caso de punto de recolección, debe mostrar dichos puntos.
- Debe ingresar al menos 10 prendas a enviar.
- *Debe enviar un email de comprobante de las prendas a vender a la empresa y al vendedor. (Puede ser otra US)*

Pruebas de usuario:

- ☐ Probar enviar prendas ingresando uno o más prendas por categoría con su descripción, una cantidad mayor a 10 prendas, y completando y forma de envío "Retiro en Domicilio" (pasa)
- ☐ Probar enviar prendas ingresando uno o más prendas por categoría con su descripción, una cantidad mayor a 10 prendas, y completando y forma de envío "Llevar a Punto de Recolección" (pasa)
- ☐ Probar enviar prendas subiendo una foto en varias prendas de tamaño menor a 512Kb (pasa)
- ☐ Probar enviar prendas subiendo una foto en una prenda con tamaño mayor a 521kb (falla)
- ☐ Probar enviar prendas sin ingresar la descripción de una prenda (falla)
- ☐ Probar enviar prendas sin ingresar la cantidad por categoría (falla)
- ☐ Probar enviar prendas con una cantidad menor a 10 prendas (falla)
- ☐ Probar enviar prendas completando los datos a un vendedor con mail válido y recibiendo el mail (pasa).

Justificación de estimación:

- **Complejidad:** Media-Alta. Requiere validaciones combinadas y varios datos ingresados y seleccionables, y adjuntar varias imágenes.
- **Esfuerzo:** Media-Alta. Se requiere esfuerzo vinculado a que es una transacción con complejidad, varias entidades a procesar, con el adjunto de una o más fotos y el envío de email.
- **Incertidumbre:** Media. El requerimiento es claro. Hay incertidumbre técnica de qué estrategia usar para subir muchas imágenes y de dónde almacenarla.

Confirmar propuesta de venta

Como Vendedor quiero confirmar la propuesta recibida para poner en venta mis prendas

Criterios de Aceptación:

- Debe ver la cantidad de prendas, fecha de recepción/envío, listado de prendas seleccionados para vender (nombre, categoría, estado, precio sugerido y ganancia).
- Debe ver, si hay prendas no seleccionadas, el listado de prendas con el motivo por el cual no fueron seleccionadas.
- Puede poner o quitar de la venta productos seleccionados.
- Debe indicar la acción a realizar (donar o retirar) de cada producto no seleccionado y/o retirado de la venta.
- Al confirmar deben quedar las prendas en condición de ser publicadas en el ecommerce.
- *Debe enviarse un email al vendedor con los pasos a seguir con las prendas a donar o con los pasos a seguir para retirar (Puede ser otra US)*

5

Pruebas de usuario:

- ☐ Probar confirmar la propuesta sin quitar prendas a vender e indicando donar los no seleccionados (pasa)
- ☐ Probar confirmar la propuesta sin prendas no seleccionadas (pasa)
- ☐ Probar confirmar la propuesta sin prendas seleccionadas y eligiendo retirar para las prendas no seleccionadas (pasa)
- ☐ Probar confirmar la propuesta con prendas no seleccionadas sin indicar acción de destino (falla)
- ☐ Probar confirmar la propuesta con prendas no seleccionadas, quitando prendas seleccionadas a vender y eligiendo como destino donar (pasa)
- ☐ Probar confirmar la propuesta con prendas seleccionadas y quitando todas de la venta, y sin productos no seleccionados (pasa)

Justificación de estimación:

- **Complejidad:** Media. Requiere mostrar un considerable volumen de información y organizarlo de manera intuitiva, tiene validaciones combinadas y varias entidades a actualizar.
- **Esfuerzo:** Media. Se requiere esfuerzo vinculado a que es una transacción con complejidad, con una interfaz de pantalla que puede llevar tiempo de diseño UX y de implementación, varias entidades a procesar, y el envío de email (si corresponde a la US).
- **Incertidumbre:** Baja. El requerimiento es claro. No hay incertidumbre técnica.

Seleccionar prenda para publicar

Como Analista de Selección quiero seleccionar las prendas que cumplan los criterios de selección para armar la propuesta de venta al vendedor

Criterios de Aceptación:

- Debe seleccionar una categoría, una marca e indicar si está seleccionada o no.
- En caso de ser seleccionada, debe seleccionar el estado de la prenda (Como nuevo, nuevo con etiqueta, ...). Si es segunda selección, debe ingresar un motivo.
- En caso de ser no seleccionada, debe ingresar un motivo.
- Debe imprimirse una etiqueta autoadhesiva con un código QR cuyo código identifica al producto.

3

(no considera impresión) u

8

(considera impresión)

Pruebas de usuario:

- ☐ Probar seleccionar una prenda completando categoría, marca, seleccionado (pasa)
- ☐ Probar seleccionar una prenda completando categoría, marca, no seleccionado y motivo de no selección (pasa)
- ☐ Probar seleccionar una prenda completando categoría, marca, no seleccionado y sin indicar un motivo de no selección (falla)
- ☐ Probar seleccionar una prenda y la impresora no tiene papel de etiquetas (falla)

Justificación de estimación: *asumiendo la impresión de la etiqueta con QR*

- **Complejidad:** Media-Alta. Requiere recuperar varias entidades, pero tiene pocos datos y validaciones simples. La complejidad estará en conectarse con la impresora de etiquetas y ajustar el formato.
- **Esfuerzo:** Media. Se requiere recuperar varias entidades complejas pero el objeto de prenda a crear es simple. La mayor parte del esfuerzo estará sujeto a la impresión de la etiqueta ya que el código QR es una librería ampliamente difundida en cualquier lenguaje.
- **Incertidumbre:** Alta. El requerimiento es claro. Hay incertidumbre técnica respecto a la conexión de la impresión para imprimir y a la generación del QR.

Completar ficha de prenda a publicar

Como Analista de Publicación quiero cargar los datos de la prenda para completar la ficha de producto a vender

5 u 8

Criterios de Aceptación:

- Debe escanear el código QR de la prenda y mostrar sus datos ya cargados.
- Debe ingresar un nombre y el peso.
- Debe completar los atributos según la categoría de la prenda.
- Debe ingresar hasta 5 fotos. Cada foto no debe superar 1Mb.
- Debe ver el precio sugerido calculado para la prenda (según categoría, marca, estado y estación) y permitir editarlo.
- Debe poder pre-visualizar la ficha de la prenda tal cual la verá el cliente.

Pruebas de usuario:

- ☐ Probar completar ficha de prenda escaneando el código QR de una prenda y completando todos los datos solicitados e ingresando 4 fotos (pasa)
- ☐ Probar completar ficha de prenda escaneando el código QR de una prenda inexistente (falla).
- ☐ Probar completar ficha de prenda sin ingresar nombre ni peso (falla)
- ☐ Probar completar ficha de prenda subiendo 1 foto mayor a 1Mb (falla)
- ☐ Probar completar ficha de prenda sin completar los atributos de la categoría (falla)
- ☐ Probar completar ficha de prenda no ingresando ninguna foto (falla)
- ☐ Probar completar ficha de prenda para una categoría, marca, estacionalidad que no tiene precio sugerido (falla)

Justificación de estimación:

- **Complejidad:** Media-Alta. Requiere interactuar con un Escáner QR (es como un teclado), mostrar los datos del producto, completar varios datos dinámicos (atributos), validaciones simples y adjuntar varias imágenes.
- **Esfuerzo:** Medio-Alto. Se requiere esfuerzo vinculado a que es una transacción con complejidad, varias entidades a procesar, con el adjunto de una o más fotos.
- **Incertidumbre:** Media-Alta. El requerimiento es claro. Hay incertidumbre técnica de cómo funciona un lector óptico de QR y de qué estrategia usar para subir muchas imágenes y de dónde almacenarla.

PRÁCTICO 4 - SCM - Herramientas de SCM (Evaluable)

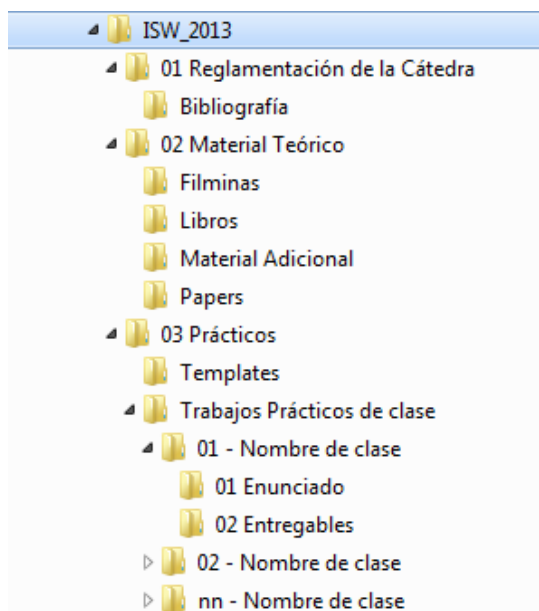
Unidad:	Unidad Nro. 3: Gestión del Software como producto
Consigna:	Comprender los conceptos de administración de configuración de software (SCM) expuestos en la clase teórica para aplicarlos en un ejercicio propuesto por la cátedra.
Objetivo:	Que el estudiante sea capaz de realizar actividades básicas de la gestión de configuración mediante el uso de una herramienta tales como la definición de una estructura de repositorio, ingreso y extracción de ítems de configuración del repositorio y definición de líneas base.
Propósito:	Aplicar los conceptos de gestión de configuración estudiados en una herramienta de software específica
Entradas:	Conceptos teóricos sobre los temas desarrollados en clase. Bibliografía referenciada sobre el tema.
Salida:	• URL de acceso al repositorio implementado • Documento con la definición de los ítems de configuración y su regla de nombrado y la definición del criterio para la creación de una línea base. Se evaluará lo siguiente: • El repositorio debe ser accesible de forma pública • Implementación de la estructura de carpetas propuesta • Los archivos se deben encontrar en la ubicación correspondiente a su definición como ítem de Configuración.
Instrucciones:	• Realizar el diseño del repositorio y reglas de nombrado de ítems de configuración • Crear un repositorio de acceso público • Crear cuentas de usuario para cada uno de los integrantes del grupo • Para el caso de estudio EcoHarmony Park, pensar en todo el material que se genera en el desarrollo de un proyecto de software en el contexto de una metodología ágil y que necesitará ser mantenido bajo control de versiones. • Implementar la estructura del repositorio propuesta para resguardar el trabajo generado durante el proyecto de desarrollo EcoHarmony Park. • Realizar Commit (colocar) de cada ítem de configuración disponible al momento. • Definir al menos un momento que considere adecuado para marcar una línea base y luego marcar la línea base definida en el repositorio.
Observaciones:	El repositorio implementado debe ser de acceso público utilizando Git o Subversion como motor de control de versiones.

SOLUCIÓN PRÁCTICO 4 - SCM

Consideraciones:

- Que sea escalable
- Contenga todos los ítems posibles de la materia
- Glosario útil
- Extensiones de los archivos a subir
- Documento de gestión en el main
- En el git, no debe estar creadas cosas que no sucedieron aun

Ejemplo de solución propuesta:



Nombre del ítem de Configuración	Regla de Nombrado	Ubicación Física	Tipo de ítem [Cátedra / Clase / Producción Propia]
Modalidad Académica	ISW_20XX_PLANIFICACIÓN.PDF	C:\Mis Documentos\UTN_FRC\4º\ISW_20XX\01 Reglamentación de la cátedra	Cátedra
Filmina	ISW_20XX_FILMINA_<MMDD>_<NOMBRE DE LA CLASE>.PDF	C:\Mis Documentos\UTN_FRC\4º\ISW_20XX\02 Material Teórico\Filminas	Clase
Lectura	ISW_20XX_<NN>_<TÍTULO DE LA LECTURA>.PDF	C:\Mis Documentos\UTN_FRC\4º\ISW_20XX\02 Material Teórico\Material Adicional	Cátedra
Paper	ISW_20XX_PAPER_<MMDD>_<NOMBRE DEL PAPER>.PDF	C:\Mis Documentos\UTN_FRC\4º\ISW_20XX\02 Material Teórico\Papers	Cátedra
Ejemplos	ISW_20XX_EJ_<MMDD>_<NN>_<NOMBRE DE LA CLASE>_<DOMINIO DEL EJEMPLO>.PDF	C:\Mis Documentos\UTN_FRC\4º\ISW_20XX\03 Prácticos\Trabajos prácticos de clase\nn <Nombre de la Clase>	Clase
Trabajo práctico de clase	ISW_20XX_TP_<MMDD>_<NN>_<NOMBRE DE LA CLASE>_<DOMINIO DEL TP>.DOCX	C:\Mis Documentos\UTN_FRC\4º\ISW_20XX\03 Prácticos\Trabajos prácticos de clase\nn <Nombre de la Clase>	Clase
Template	ISW_20XX_TEMPLATE_<Tema>.PDF	C:\Mis Documentos\UTN_FRC\4º\ISW_20XX\03 Prácticos\Templates	Cátedra
Resolución de trabajo práctico	ISW_20XX_RES_TP_<MMDD>_<NN>_<NOMBRE DE LA CLASE>_<DOMINIO DEL TP>.DOCX	C:\Mis Documentos\UTN_FRC\4º\ISW_20XX\03 Prácticos\Trabajos prácticos de clase\nn <Nombre de la Clase>\02 Entregables	Producción Propia

Universidad Tecnológica Nacional – Facultad Regional Córdoba
 Cátedra de Ingeniería y Calidad de Software – 2026
 Curso: 4K1 - 4k2 - 4k3

Nombre del Ítem de Configuración	Regla de Nombrado	Ubicación Física	Tipo de Ítem [Cátedra / Clase / Producción Propia]
Enunciado de Trabajo Práctico	ISW_20XX_TP_<MMDD>_<NN>_ENUNCIADO.PDF	C:\Mis Documentos\UTN_FRC\4º\ISW_20XX\03 Prácticos\Trabajos Prácticos de Clase\nn <Nombre de la Clase>\01 Enunciado	Cátedra

Sigla	Significado
<NN>	Número de orden correlativo
<MMDD>	Fecha en formato numérico de la clase (MesDía).
<NOMBRE DE LA CLASE>	Nombre de la clase, el cual se indica por el tema que la misma abarca. Ejemplos son: Estimaciones, Riesgos, Métricas.
<NOMBRE DEL PAPER>	Nombre del Paper brindado por la Cátedra como bibliografía.
<DOMINIO DEL TP>	Nombre del dominio del que trata el trabajo práctico de los dominios que se trabajan durante el semestre. Ejemplo: Seguimiento de Defectos.
<DOMINIO DEL EJEMPLO>	Nombre del dominio del ejemplo dado.
<Tema>	Nombre del tema al cual pertenece el template.

PRÁCTICO 5 - SCM - Uso del Repositorio (Evaluable)

Unidad:	Unidad Nro. 3: Gestión del Software como producto
Consigna:	Comprender los conceptos de administración de configuración de software (SCM) expuestos en la clase teórica para aplicarlos en un ejercicio propuesto por la cátedra.
Objetivo:	Que el estudiante sea capaz de realizar actividades básicas de la gestión de configuración de software, utilizando una herramienta automatizada. Tales actividades son: la definición de una estructura de repositorio, ingreso y extracción de ítems de configuración del repositorio y definición de líneas base.
Propósito:	Aplicar los conceptos de gestión de configuración de software estudiados, utilizando una herramienta específica
Entradas:	Conceptos teóricos sobre los temas desarrollados en clase. Bibliografía referenciada sobre el tema. Estructura del Repositorio, Plantilla de Ítems de Configuración, URL y Credenciales de acceso del repositorio implementado en el Trabajo Práctico 4
Salida:	Se evaluará que: ● Cada Integrante del Grupo ha realizado operaciones de <i>Commit</i> (colocar) y <i>Update</i> (obtener) sobre el proyecto de desarrollo de software para el cursado de la materia. ● Los ítems de configuración disponibles presentan una evolución acorde a su contenido a lo largo del cursado con participación de los integrantes del grupo. ● Existe al menos una Línea Base creada en el repositorio según el criterio definido en el Trabajo Práctico 4. ● Que los ítems de configuración respeten la ubicación propuesta en la estructura del repositorio.
Instrucciones:	● Realizar <i>Commit</i> (colocar) de cada ítem de configuración disponible al momento con las cuentas de usuario de cada integrante del grupo. ● Generar una Línea Base según criterio definido.
Observaciones:	Las operaciones de gestión de configuración deben realizarse a lo largo del cursado de la materia, a medida que evoluciona el proyecto de software.

PRÁCTICO 6 - TDD (Evaluable)

Unidad:	Unidad Nro. 4: Aseguramiento de Calidad de Proceso y de Producto
Consigna:	Implementar una funcionalidad determinada utilizando el enfoque de <i>Desarrollo Conducido por Pruebas (TDD)</i> , en el lenguaje de programación elegido por el grupo, utilizando el enfoque Red-Green-Refactor para construir el código desde las pruebas.
Objetivo:	Aplicar el enfoque TDD para diseñar y construir software a partir de pruebas automatizadas, poniendo en práctica buenas prácticas de diseño y desarrollo orientado a la calidad desde el inicio.
Propósito:	Familiarizarse con el ciclo TDD (Red-Green-Refactor), incorporando buenas prácticas de diseño y testing, y fortaleciendo la disciplina del desarrollo guiado por pruebas como base para un código de calidad.
Entradas:	● Conceptos teóricos sobre el tema desarrollados en clase. ● Bibliografía referenciada sobre el tema. ● Enunciado del Trabajo Práctico 2: EcoHarmony Park ● Definición completa de las User Stories correspondientes al Trabajo Práctico 2 “Requerimientos Ágiles – User Stories y Estimaciones” ● Reglas de estilo de código seleccionadas por el grupo para el Trabajo Práctico 2
Salida:	● Implementación funcional de la User Story asignada ● Conjunto de pruebas automatizadas que cubren los distintos casos definidos. ● Documento que justifique las decisiones de diseño tomadas.
Instrucciones:	● Seleccionar una User Story a implementar de entre las siguientes opciones: ○ <i>Comprar entradas (grupos impares)</i> ○ <i>Inscribirme a actividad (grupos pares)</i> ● Utilizando el lenguaje y documento de estilos de código seleccionado para el Trabajo Práctico 2 “Requerimientos Ágiles – User Stories y Estimaciones”, implementar la funcionalidad aplicando el ciclo TDD: ○ Red: Escribir una prueba que falle. ○ Green: Implementar el código mínimo para que pase. ○ Refactor: Mejorar el diseño del código sin romper las pruebas.

EcoHarmony Park: Descripción del dominio

Los dueños del bioparque “EcoHarmony Park” están evaluando el desarrollo de una aplicación móvil (Android, iOS) ya que la cantidad de visitantes anuales aumentaron significativamente al igual que los avances tecnológicos. La misma tiene como objetivo mejorar la experiencia de los visitantes al proporcionar información útil y enriquecedora sobre las exhibiciones, los horarios de alimentación de los animales, los senderos para caminar y otros aspectos del parque como la autogestión al momento de realizar la compra de entradas.

Hemos tenido el privilegio de entablar una conversación con Miguel Rodríguez, uno de los propietarios de EcoHarmony Park, y a continuación, compartimos los detalles de la interacción:

El Doce: ¡Hola! Estoy aquí en el hermoso bioparque "EcoHarmony Park" con el Sr. Rodríguez, para hablar sobre una emocionante novedad: ¡una nueva aplicación móvil que mejorará la experiencia de los visitantes aquí en el parque! ¿Me podrías contar brevemente en qué consiste la aplicación?

Miguel: ¡Hola! Gracias por tomarte el tiempo para hablar conmigo. Estoy encantado de compartir más detalles sobre la idea de nuestra aplicación. En primer lugar, ofrecerá a los visitantes mapas interactivos que les ayudarán a explorar el parque de manera más eficiente. Asimismo, proporcionará información detallada sobre cada exhibición y horario de alimentación de cada especie que cuidamos. También se podrá autogestionar la compra de entradas al parque, pudiendo realizar directamente el pago a través de la pasarela de Mercado Pago, lo que permitirá agilizar el ingreso. Por último, los visitantes pueden inscribirse a las diferentes actividades diarias que posee el parque.

El Doce: Wow! Que interesante ¿Podrías explicarme cómo puede un visitante acceder a los horarios de alimentación a través de la aplicación?

Miguel: ¡Claro! Una vez que descarguen la aplicación y la abran, encontrarán una sección dedicada a "Horarios de Alimentación disponibles". Cada horario estará detallado de manera clara, indicando la especie, nombre del animal, edad, hora programada, sector del parque (terrestres, acuáticos o aéreos) y cuidador encargado (nombre y apellido). Por otra parte, aquellos que estén próximos a comenzar (faltando una hora) los mostraremos con un mensaje que diga: ¡Apúrate que ya comienza!

Es importante considerar que los horarios se mostrarán en orden, comenzando con los más próximos a la fecha actual y extendiéndose hasta una semana en el futuro. Para que sea más amigable la interfaz pensamos en proporcionar la posibilidad de filtrar por la fecha en la que el visitante estará en el parque y que se visualicen únicamente los horarios correspondientes a la misma.

Además, en una versión posterior, los visitantes podrán configurar notificaciones para recibir recordatorios de los horarios de alimentación que deseen seguir de cerca.

El Doce: ¡Eso suena muy útil! ¡Al igual que el mapa interactivo! ¿Podrías comentarme cuáles serán los elementos y características del parque que estarán disponibles para visualizar en él? ¿Qué acciones podrán llevar a cabo los visitantes a través de esta herramienta?

Miguel: Este mapa ofrece una vista panorámica de todo el parque, dividido en sectores diferenciados por colores. Asimismo, el mapa contiene iconos informativos: “Inodoro” para identificar los baños distribuidos en el parque, una “Bolsa” para indicar los puntos de venta, y un “Micrófono” para el show especial del día. Los visitantes pueden hacer click en este último icono para obtener el nombre del show y su horario, en caso de que haya un show programado. En una futura mejora pensamos incluir en este mapa la ubicación del visitante en tiempo real.

El Doce: Para alguien como yo, con un sentido de la orientación no muy desarrollado, suena como una herramienta imprescindible, ¡jaja! ¿Hay otros aspectos del parque que también planean ofrecer a través de la aplicación? ¿Habías mencionado la inscripción a las actividades, no es cierto?

Miguel: Sí! me estaba olvidando de comentarlo. El parque ofrece la posibilidad de realizar actividades de una hora sumamente enriquecedoras para los visitantes, entre las que se encuentran “Tirolesa”, “Safari”, “Palestra” y “Jardinería”. Para poder realizar la inscripción deben seleccionar del conjunto de actividades la que desean realizar, eligiendo en primer lugar el horario y completando luego los datos del visitante: nombre, DNI, edad y talla de vestimenta si la actividad lo demanda. Finalmente, para concluir el proceso de inscripción, pedimos a los visitantes que acepten los términos y condiciones específicos de la actividad en la que participarán. Nos interesa gestionar la inscripción de las actividades porque tienen cupos limitados y es muy útil para planificarlas con anterioridad según los inscriptos que tengamos.

El Doce: Para finalizar esta pequeña entrevista, ¿Podrías explicarme el proceso de compra de entradas a través de la app?

Miguel: ¡Pero claro! Para realizar la compra de entradas el visitante debe registrarse en la aplicación, ingresando un mail y una contraseña. A continuación, debe ingresar a la sección “Comprar entradas” y allí indicar la fecha de visita deseada, la cantidad de entradas requeridas (que no puede superar las 10) y la edad de cada visitante. Finalmente, el sistema mostrará el monto total y se debe seleccionar la forma de pago: efectivo en caso de querer pagar en boletería o con tarjeta, donde se redirigirá al usuario a la página de Mercado Pago para completar el proceso de forma segura. Finalmente, recibirán un mensaje de confirmación vía mail y sus entradas serán verificadas al momento de ingresar al parque. La posibilidad de comprar las entradas mediante la aplicación nos parece fundamental para comenzar a probar la recepción de la aplicación, incluyendo el pago electrónico ya que creemos que será el más usado.

El Doce: ¡Muy claro! Parece que la aplicación requiere mucha atención. ¿Tienen previsto contratar personal adicional para mantener actualizados los horarios, las actividades y todo lo que implica?

Miguel: No, no hay problema en ese aspecto. Planeamos gestionar esa tarea por fuera de la aplicación, a través de nuestro equipo de administradores, que se encargará de mantener todo actualizado.

El Doce: Ah, entiendo. ¡Gracias por compartir todos estos emocionantes detalles! Más adelante coordinamos otra nota para ver esta aplicación en funcionamiento.

User stories:

Comprar entradas	8
COMO visitante QUIERO comprar una entrada PARA asegurar mi visita al parque	
Criterios de Aceptación: ● Debe indicar la fecha de visita deseada, la cantidad de entradas requeridas, la edad de cada visitante y tipo de pase (VIP o regular). ● La fecha de visita guiada puede ser del día actual o futuro. ● Debe enviar un mensaje de confirmación vía mail ● Debe redirigir a mercado pago al confirmar la compra si el pago es con tarjeta de crédito ● La fecha de la visita debe estar dentro de los días en que el parque está abierto. ● Debe seleccionar la forma de pago: efectivo en caso de querer pagar en boletería o con tarjeta	

- La cantidad de entradas requeridas no debe ser mayor a 10
- Al finalizar la compra se debe informar la cantidad de entradas compradas y la fecha.
- Se debe permitir la compra de entradas solo a usuarios registrados.

Pruebas de usuario:

- Probar comprar una entrada indicando la fecha de visita dentro de los días disponibles, una cantidad de entradas menor a 10, la edad de todos los visitantes, el tipo de pase, la forma de pago con tarjeta mediante Mercado pago y recepción del mail de confirmación (pasa)
- Probar comprar entradas sin seleccionar forma de pago (falla)
- Probar comprar entradas ingresando una fecha de visita en la cual el parque se encuentra cerrado (falla)
- Probar comprar entradas ingresando una cantidad de entradas mayor a 10 (falla)

TESTS:

```
def test_generar_mail_success():
    # Precondiciones
    usuario = {
        "nombre": "Ravyn",
        "apellido": "Lenae",
        "permisos": "Visitante"}
    usuario = iniciar_sesion("visitante", "clave_visitante")
    compra = {
        "fecha_visita": "10-10-2025",
        "cantidad_de_entradas": "1",
        "edades": "[26]",
        "tipo_pase": "[regular]",
        "forma_de_pago": "Efectivo",
        "monto": ""}

    # Secuencia
    seleccionar_boton('Confirmar compra')
    comprar_entradas(compra)
    monto = calcular_monto(edades = compra["edades"], tipos_pase =
compra["tipo_pase"])
    resultado_esperado = generar_mail(fecha = compra["fecha"],
cantidad_entradas = compra["cantidad_de_entradas"], tipo_pase =
compra["tipo_pase"], monto = monto, forma_pago = compra["forma_de_pago"])

    # Resultados
    assertEquals(resultado_esperado == "Compra de entradas verificada !
Se ha comprado una entrada/s de tipo regular para el día 10/10/2025. El
costo total es de $10.000. Debe abonarlas al llegar a boletería. Los
esperamos !", "Error al intentar generar el mail")

    # En caso de que la prueba pase
    print("La prueba para validar que se haya generado el mail con los
datos correctos ha pasado")
```

```
def test_comprar_entrada_sin_ingresar_forma_de_pago_fail():
    # Precondiciones
    usuario = {"nombre": "Taylor", "apellido": "Swift", "permisos":
"Visitante"}
    usuario = iniciar_sesion("visitante", "clave_visitante")
    compra = {
        "fecha": "9/10/2025",
        "cantidad_entradas": "1",
        "edades": "[35]",
        "tipo_pase": "[VIP]",
        "forma_de_pago": "",
        "monto": ""}

    # Secuencia
    seleccionar_boton('Confirmar compra')
    confirmar_compra(compra)
    compra["monto"] = calcular_monto(edades = compra["edades"],
tipos_pase = compra["tipo_pase"])
    resultado_esperado = comprar_entrada(compra, usuario,
dias_cerrados=[])

    # Resultados
    assertEquals(resultado_esperado == "No se ha seleccionado un tipo de
pago. Por favor seleccione uno", "Se debe informar que no se seleccionó una
forma de pago")

    # En caso de que la prueba pase
    print("La prueba para validar que se haya seleccionado un tipo de
pago ha pasado")
```

```
def test_comprar_entrada_fecha_parque_cerrado_fail():
    # Precondiciones
    usuario = {"nombre": "Charli", "apellido": "XCX", "permisos":
"visitante"}
    usuario = iniciar_sesion("visitante", "clave_visitante")
    compra = {
        "fecha": "10/10/2025",
        "cantidad_entradas": "3",
        "edades": "[33, 35, 31]",
        "tipo_pase": "[\"VIP\", \"VIP\", \"VIP\"]",
        "forma_de_pago": "Boletería",
        "monto": ""}
    dias_cerrados = ["martes", "jueves"]

    # Secuencia
    seleccionar_boton('Confirmar compra')
    confirmar_compra(compra)
```

```
    compra["monto"] = calcular_monto(edades = compra["edades"],
tipos_pase = compra["tipo_pase"])
    resultado_esperado =

    # Resultados
    try:
        comprar_entrada(compra, usuario, dias_cerrados)
    except (ExceptionType FechaCerradaex):
        assertEquals(ex['message'], "El parque está cerrado en la fecha
seleccionada.")

    # En caso de que la prueba pase
    print("La prueba para validar que se haya seleccionado una fecha en
la cual el parque esté abierto pasa")
```

```
def test_comprar_entrada_por_mp_success():
    # Precondiciones
    usuario = {"nombre": "Rigoberta", "apellido": "Badidni", "permisos":
"Visitante"}
    usuario = iniciar_sesion("visitante", "clave_visitante")
    compra = {"fecha": "14/10/2025", "cantidad_entradas": "2", "edades":
"[35, 35]", "tipo_pase": "[VIP, VIP]", "forma_de_pago": "Tarjeta", "monto":
""}

    // Secuencia
    seleccionar_boton('Confirmar compra')
    confirmar_compra(compra)
    compra["monto"] = calcular_monto(edades = compra["edades"],
tipos_pase = compra["tipo_pase"])
    resultado_esperado = comprar_entrada(compra, usuario,
dias_cerrado=[])

    # Resultados
    assertEquals(resultado_esperado == "Se ha redirigido a Mercado Pago
de forma exitosa", "No se ha podido redirigir a Mercado Pago")

    # En caso de que la prueba pase
    print("La prueba para validar que se redirija a Mercado Pago pasa")
```

```
def test_comprar_entradas_con_cantidad_mayor_a_10_fail():
    # Precondiciones
    usuario = {'mail': 'visitante@dominio.com.ar', 'clave': '*****'}
    usuario = iniciar_sesion('visitante', 'clave_visitante')
    datos_compra_a_realizar = {
        'fecha_visita': '1-10-2025',
        'cantidad_de_entradas': '11',
        'edad_de_visitantes': [10, 8, 25, 32, 77, 81, 43, 17, 19, 6],
        'tipo_de_pase': 'Regular',
        'forma_de_pago': 'Efectivo'
    }

    # Secuencia
    seleccionar_boton('Confirmar compra')
    confirmar_compra(compra)
    compra["monto"] = calcular_monto(edades = compra["edades"],
    tipos_pase = compra["tipo_pase"])

    # Resultados
    try:
        comprar_entradas(datos_compra_a_realizar, usuario, dias_cerrados=[])
    except (ExceptionType CantidadEntradas ex):
        assertEquals(ex['message'], "La cantidad de entradas debe ser un
    número entero entre 1 y 10")

    # PRINT en caso de que la prueba pase
    print("La prueba para validar que la cantidad de entradas en una compra
    sea correcta ha pasado")
```

```
def test_comprar_entradas_success():
    # Precondiciones
    usuario = {'mail': 'visitante@dominio.com.ar', 'clave': '*****'}
    usuario = iniciar_sesion('visitante', 'clave_')
    compra_a_realizar = {
        'fecha_visita': '01-10-2025',
        'cantidad_de_entradas': '5',
        'edad_de_visitantes': [10, 8, 25, 32, 77],
        'tipo_de_pase': 'Regular',
        'forma_de_pago': 'Tarjeta'
    }
    dias_no_disponibles = ['martes', 'jueves']

    # Secuencia
    seleccionar_boton('Confirmar compra')
    confirmar_compra(compra)
    compra["monto"] = calcular_monto(edades = compra["edades"],
    tipos_pase = compra["tipo_pase"])
```

```
compra, confirmacion = comprar_entrada(compra_a_realizar, usuario,
dias_no_disponibles)

# Resultado
assertEquals(compra.objectType == Compra, "Al confirmar la compra se
debe crear un objeto de tipo Compra")
assertEquals(confirmacion == "Compra generada con éxito: Fecha: 01-10-
2025 | Cantidad de entradas compradas: 5", "No se informa la compra de
entradas, o el mensaje es erróneo")
assertEquals(compra.fecha_visita == 01-10-2025, "Las fechas de visita
no coinciden")
assertEquals(compra.cantidad_de_entradas == 5, "La cantidad de entradas
compradas no coinciden")
assertAll(edad in compra_a_realizar["edad_de_visitantes"] for edad in
compra.edad_de_visitantes, "Las edades de la compra creada no coinciden con
las indicadas")
assertEquals(compra.tipo_de_pase == "Regular", "La cantidad de entradas
compradas no coinciden")
assertEquals(compra.forma_de_pago == "Tarjeta", "La forma de pago de la
compra creada no coincide con la indicada")

# PRINT si todos los casos son verdaderos
print("La prueba para validar comprar una entrada ha pasado")
```

```
def test_comprar_entradas_objeto_compra_null_fail():
# Precondiciones
usuario = {'mail': 'visitante@dominio.com.ar', 'clave': '*****'}
usuario = iniciar_sesion('visitante', 'clave_')
compra_a_realizar = {
    'fecha_visita': '01-10-2025',
    'cantidad_de_entradas': '5',
    'edad_de_visitantes': "",
    'tipo_de_pase': 'Regular',
    'forma_de_pago': 'Tarjeta'
}
dias_no_disponibles = ['martes', 'jueves']

# Secuencia
seleccionar_boton('Confirmar compra')
confirmar_compra(compra)
compra["monto"] = calcular_monto(edades = compra["edades"], tipos_pase
= compra["tipo_pase"])

# Resultado
try:
    compra, confirmacion = comprar_entrada(compra_a_realizar, usuario,
dias_no_disponibles)
except (ExceptionType CompraFallida ex):
```

```
assertEquals(ex["message"], "La compra no se pudo completar.")

# PRINT si todos los casos son verdaderos
print("La prueba para validar comprar una entrada ha pasado")
```

Pseudocódigo función principal y auxiliares:

```
def comprar_entradas(datos_compra, usuario):
    # Validacion por forma de pago
    if not validar_forma_pago(datos_compra["forma_de_pago"]):
        return "No se ha seleccionado un tipo de pago. Por favor
seleccione uno"

    # Validacion por fecha
    if not validar_fecha_parque.fecha, dias_cerrado):
        throw Exception FechaCerrada("El parque está cerrado en la fecha
seleccionada.")

    # Validacion por cant de entradas
    if not validar_cantidad_entradas(datos_compra['cantidad_entradas']):
        throw Exception CantidadEntradas("La cantidad de entradas debe ser
un número entero entre 1 y 10")

    # Confirmacion y Generar Mail
    if datos_compra["forma_de_pago"] == "Tarjeta":
        redirigir_mercado_pago(datos_compra["monto"])

    generar_mail(datos_compra["fecha"], datos_compra["cantidad_entradas"],
datos_compra["tipo_pase"], datos_compra["monto"],
datos_compra["forma_pago"])
    # Crea la compra y muestra el mensaje de confirmación
    compra_realizada = new Compra(datos_compra)
    mensaje_confirmacion = mostrar_confirmacion(compra_realizada)

    return compra_realizada, mensaje_confirmacion
```

```
def validar_forma_pago(forma_pago):
    return forma_pago in ("Tarjeta", "Efectivo")
```

```
def validar_fecha_parque(fecha_visita, dias_cerrado):
    return not fecha_visita.day in dias_cerrado
```

```
def validar_cantidad_entradas(cantidad_entradas):
    return cantidad_entradas <= 10 or cantidad_entradas > 0
```

```
def generar_mail(fecha, cantidad_entradas, tipo_pase, monto, forma_pago)
    return f"Compra de entradas verificada ! Se ha comprado {"una" if
len(cantidad_entradas) == 1 else "cantidad_entradas"} entrada/s de tipo
{tipo_pase} para el día {fecha}. El costo total es de ${monto}. "Debe
abonarlas al llegar a boletería" if forma_pago == "Efectivo" else "El monto
ya se ha cobrado a través de su tarjeta". Los esperamos ! "
```

```
def redirigir_mercado_pago(monto):
    conexion_pasarela_pagos_mercado_pago = sendRequest(monto)
    return "Se ha redirigido a Mercado Pago de forma exitosa" if
conexion_pasarela_pagos_mercado_pago == "True" else "Error al intentar
redirigir a Mercado Pago"
```

```
def mostrar_confirmacion(compra):
    if compra is not null:
        return f"Compra generada con éxito: Fecha: {compra.fecha_visita}
| Cantidad de entradas compradas: {compra.cantidad_de_entradas}"
    else:
        throw Exception CompraFallida("La compra no se pudo completar.")
```

Inscribirme a actividad

COMO visitante QUIERO inscribirme a una actividad PARA reservar mi lugar en la misma.

3

Criterios de Aceptación:

- Debe requerir seleccionar una actividad del conjunto de actividades de la lista de "Tirolesa", "Safari", "Palestra" y "Jardinería", siempre y cuando tengan cupos disponibles para el horario seleccionado
- Debe requerir seleccionar el horario dentro de los disponibles
- Debe indicar la cantidad de personas que participaran de la actividad
- Para cada persona que participa, debe ingresar los datos del visitante: nombre, DNI, edad y talla de vestimenta si la actividad lo demanda
- Debe requerir aceptar los términos y condiciones específicos de la actividad en la que participarán.
- Al finalizar la inscripción, se debe enviar un correo electrónico con información de la inscripción realizada. Debe contener un código QR.

Pruebas de usuario:

- Probar inscribirse a una actividad del listado que poseen cupos disponibles, seleccionando un horario, ingresando los datos del visitante (nombre, DNI, edad, talla de la vestimenta si la actividad lo requiere) y aceptando los términos y condiciones (pasa)
- Probar inscribirse a una actividad que no tiene cupo para el horario seleccionado (falla)
- Probar inscribirse a una actividad sin ingresar talla de vestimenta porque la actividad no lo requiere (pasa)
- Probar inscribirse a una actividad seleccionando un horario en el cual el parque está cerrado o la actividad no está disponible (falla)
- Probar inscribirse a una actividad sin aceptar los términos y condiciones de la actividad (falla)
- Probar inscribirse a una actividad sin ingresar el talla de la vestimenta requerido por la actividad (falla)

TESTS:

```
def test_inscribir_actividad_sin_terminos_fail():
    # Precondiciones
```

```
    usuario = {"nombre": "Pink", "apellido": "Pantheress", "permisos":  
"Visitante"}  
    usuario = iniciar_sesion("visitante", "clave_visitante")  
    actividades_a_realizar = [  
    'actividad':{  
        'tipo': 'Safari',  
        'duracion': 30,  
        'horarios': ['10:00', '11:00', '12:00']  
        'cupo_total': 15,  
        'cupos_restantes_por_fecha': [  
            {  
                'fecha': '01-10-2025',  
                'cupos': [  
                    {'horario': '10:00', 'cupo_restante': 0},  
                    {'horario': '11:00', 'cupo_restante': 1},  
                    {'horario': '12:00', 'cupo_restante': 14},  
                ]  
            },  
            {  
                'fecha': '02-10-2025',  
                'cupos': [  
                    {'horario': '10:00', 'cupo_restante': 15},  
                    {'horario': '11:00', 'cupo_restante': 15},  
                    {'horario': '12:00', 'cupo_restante': 15},  
                ]  
            }  
        ],  
        'fechas_actividad': ['01-10-2025', '02-10-2025', '03-10-2025'],  
        'requiere_vestimenta': false  
    },  
    'actividad'{  
        'tipo': 'Palestra',  
        'duracion': '45',  
        'horarios': ['10:00', '11:00', '12:00']  
        'cupo': 20,  
        'cupos_restantes': [20, 19, 14],  
        'fechas_actividad': ['01-10-2025', '02-10-2025', '03-10-2025'],  
        'requiere_vestimenta': true  
    }  
    ]  
  
    inscripcion = {  
        "horario": "10/10/2025:18:30:00",  
        "cantidad": "2",  
        "actividad": "Tiroleza",  
        "participantes": [  
            {"nombre": "Pink",  
            "DNI": "20300666",
```



```
        "edad": "24",
        "talla": "",
        {"nombre": "Lorde",
         "DNI": "39564789",
         "edad": "28",
         "talla": ""}],
        "terminos": "False"}
    }
    # Secuencia
    seleccionar_boton('Confirmar Inscripción')
    resultado_esperado = inscribirse_a_actividad(inscripcion,
    actividades_a_realizar)

    # Resultados
    assertEquals(resultado_esperado == "No han aceptado los términos y
    condiciones. Por favor, acéptelos para poder inscribirse a la actividad",
    "Se debe informar que no se aceptaron los términos y condiciones.")

    # En el caso de que la prueba pase
    print("La prueba que valida el rechazo de la inscripción en caso de
    que no se acepten los términos y condiciones pasa")
```

```
def test_inscribir_actividad_sin_talle_requerido_fail():
    # Precondiciones
    usuario = {"nombre": "FKA", "apellido": "Twigs", "permisos": "Visitante"}
    usuario = iniciar_sesion("visitante", "clave_visitante")
    actividades_a_realizar = [
        'actividad':{
            'tipo': 'Safari',
            'duracion': 30,
            'horarios': ['10:00', '11:00', '12:00']
            'cupos_total': 15,
            'cupos_restantes_por_fecha': [
                {
                    'fecha': '01-10-2025',
                    'cupos': [
                        {'horario': '10:00', 'cupos_restante': 0},
                        {'horario': '11:00', 'cupos_restante': 1},
                        {'horario': '12:00', 'cupos_restante': 14},
                    ]
                },
                {
                    'fecha': '02-10-2025',
                    'cupos': [
                        {'horario': '10:00', 'cupos_restante': 15},
                        {'horario': '11:00', 'cupos_restante': 15},
                        {'horario': '12:00', 'cupos_restante': 15},
                    ]
                }
            ]
        }
    ]
```

```
    ]
    }
  ],
  'fechas_actividad': ['01-10-2025', '02-10-2025', '03-10-2025'],
  'requiere_vestimenta': false
},
'actividad'{
  'tipo': 'Palestra',
  'duracion': '45',
  'horarios': ['10:00', '11:00', '12:00']
  'cupos': 20,
  'cupos_restantes': [20, 19, 14],
  'fechas_actividad': ['01-10-2025', '02-10-2025', '03-10-2025'],
  'requiere_vestimenta': true
}
]

inscripcion = {
  "horario": "10/10/2025:18:30:00",
  "cantidad": "1",
  "actividad": "Palestra",
  "participantes": [
    {"nombre": "Pink",
      "DNI": "20300666",
      "edad": "24",
      "talla": ""}],
  "terminos": "True"}

# Secuencia
seleccionar_boton('Confirmar Inscripción')
resultado_esperado = inscribirse_a_actividad(inscripcion,
actividades_a_realizar)

# Resultados
assertEquals(resultado_esperado == "No se indicó el talle requerido
para poder realizar la actividad", "Se debe informar que no se ingresó la
talla requerida por la actividad.")

# En el caso de que la prueba pase
print("La prueba que valida que se haya seleccionado una talla para
una actividad que requiera talla pasa")
```

```
def test_inscribir_actividad_formato_dni_incorrecto_fail():
  # Precondiciones
  usuario = {"nombre": "Tyler", "apellido": "TheCreator", "permisos":
"Visitante"}
  usuario = iniciar_sesion("visitante", "clave_visitante")
  actividades_a_realizar = [
```

```
'actividad':{
  'tipo': 'Safari',
  'duracion': 30,
  'horarios': ['10:00', '11:00', '12:00']
  'cupos_total': 15,
  'cupos_restantes_por_fecha': [
    {
      'fecha': '01-10-2025',
      'cupos': [
        {'horario': '10:00', 'cupos_restante': 0},
        {'horario': '11:00', 'cupos_restante': 1},
        {'horario': '12:00', 'cupos_restante': 14},
      ]
    },
    {
      'fecha': '02-10-2025',
      'cupos': [
        {'horario': '10:00', 'cupos_restante': 15},
        {'horario': '11:00', 'cupos_restante': 15},
        {'horario': '12:00', 'cupos_restante': 15},
      ]
    }
  ],
  'fechas_actividad': ['01-10-2025', '02-10-2025', '03-10-2025'],
  'requiere_vestimenta': false
},
'actividad'{
  'tipo': 'Palestra',
  'duracion': '45',
  'horarios': ['10:00', '11:00', '12:00']
  'cupos': 20,
  'cupos_restantes': [20, 19, 14],
  'fechas_actividad': ['01-10-2025', '02-10-2025', '03-10-2025'],
  'requiere_vestimenta': true
}
]
inscripcion = {
  "horario": "10/10/2025:18:30:00",
  "cantidad": "1",
  "actividad": "Tirolesa",
  "participantes": [
    {"nombre": "Tyler",
      "DNI": "Labubu",
      "edad": "34",
      "talla": ""}],
  "terminos": "True"}
```

Secuencia

```
seleccionar_boton('Confirmar Inscripción')

# Resultados
try:
    resultado_esperado = inscribirse_a_actividad(inscripcion,
    actividades_a_realizar)
except (ExceptionType DocumentoInvalido ex):
    assertEquals(ex["message"] == "DNI en formato no numérico", "Se
    debe informar si el formato del DNI ingresado es válido")

# En el caso de que la prueba pase
print("La prueba que valida el formato del DNI pasa")
```

```
def test_inscribirse_a_actividad_sin_cupo_para_horario_fail():
    # Precondiciones
    tipos_de_actividades = ['Safari', 'Palestra', 'Jardinería', 'Tirolesa']
    actividades_a_realizar = [
        'actividad':{
            'tipo': 'Safari',
            'duracion': 30,
            'horarios': ['10:00', '11:00', '12:00']
            'cupos_total': 15,
            'cupos_restantes_por_fecha': [
                {
                    'fecha': '01-10-2025',
                    'cupos': [
                        {'horario': '10:00', 'cupo_restante': 0},
                        {'horario': '11:00', 'cupo_restante': 1},
                        {'horario': '12:00', 'cupo_restante': 14},
                    ]
                },
                {
                    'fecha': '02-10-2025',
                    'cupos': [
                        {'horario': '10:00', 'cupo_restante': 15},
                        {'horario': '11:00', 'cupo_restante': 15},
                        {'horario': '12:00', 'cupo_restante': 15},
                    ]
                }
            ],
            'fechas_actividad': ['01-10-2025', '02-10-2025', '03-10-2025'],
            'requiere_vestimenta': false
        },
        'actividad'{
            'tipo': 'Palestra',
```

```
        'duracion': '45',
        'horarios': ['10:00', '11:00', '12:00']
        'cupo': 20,
        'cupos_restantes': [20, 19, 14],
        'fechas_actividad': ['01-10-2025', '02-10-2025', '03-10-2025'],
        'requiere_vestimenta': true
    }
]
# dias_no_disponibles = ['29-09-2025', '30-09-2025', '5-10-2025', '6-
10-2025']
inscripcion = {
    "horario": "1/10/2025:10:00:00",
    "cantidad": "1",
    "actividad": "Safari",
    "participantes": [
        {'nombre': 'Ezequiel Gustavo Izaguirre',
        'dni': '43997008',
        'edad': '23',
        'talla_vestimenta': ""}],
    "terminos": "True"}

# Secuencia
seleccionar_boton('Inscribirme a actividad')
# Llamado a la US
inscribir_a_actividad(inscripcion = 'inscripcion', actividades =
'actividades_a_realizar')

# Resultados
inscripcion_realizada = obtener_inscripcion_realizada()
resultado_esperado = inscribirse_a_actividad(inscripcion,
actividades_a_realizar)

assertEquals(resutlado_esperado == "No hay cupos para ese dia y horario
seleccionado", "No se informa la falta de cupos para el dia y horario
seleccionado")

# Print si el caso de prueba pasa
print("La prueba para validar que se inscriba en una actividad con cupo
disponible pasa")
```

```
def test_inscribirse_a_actividad_parque_cerrado_fail():
    # Precondiciones
    tipos_de_actividades = ['Safari', 'Palestra', 'Jardinería', 'Tirolesa']
    actividades_a_realizar = [
        'actividad':{
            'tipo': 'Safari',
            'duracion': 30,
```

```
'horarios': ['10:00', '11:00', '12:00']
'cupo_total': 15,
'cupos_restantes_por_fecha': [
  {
    'fecha': '01-10-2025',
    'cupos': [
      {'horario': '10:00', 'cupo_restante': 0},
      {'horario': '11:00', 'cupo_restante': 1},
      {'horario': '12:00', 'cupo_restante': 14},
    ]
  },
  {
    'fecha': '02-10-2025',
    'cupos': [
      {'horario': '10:00', 'cupo_restante': 15},
      {'horario': '11:00', 'cupo_restante': 15},
      {'horario': '12:00', 'cupo_restante': 15},
    ]
  }
],
'fechas_actividad': ['01-10-2025', '02-10-2025', '03-10-2025'],
'requiere_vestimenta': false
},
'actividad'{
  'tipo': 'Palestra',
  'duracion': '45',
  'horarios': ['10:00', '11:00', '12:00']
  'cupo': 20,
  'cupos_restantes': [20, 19, 14],
  'fechas_actividad': ['01-10-2025', '02-10-2025', '03-10-2025'],
  'requiere_vestimenta': true
}
]
dias_no_disponibles = ["martes","jueves"]
inscripcion = {
  "horario": "9/10/2025:18:30:00",
  "cantidad": "1",
  "actividad": "Tiroleza",
  "participantes": [
    {'nombre': 'Ezequiel Gustavo Izaguirre',
     'dni': '43997008',
     'edad': '23',
     'talla_vestimenta': ""}],
  "terminos": "True"}
}

# Secuencia
seleccionar_boton('Inscribirme a actividad')
```

```
# Llamado a la US
resultado = inscribirse_a_actividad(inscripcion,
actividades_a_realizar)

# Resultados
assertEquals (resultado == "El parque se encuentra cerrado en el día y
horario seleccionado", "Se debe informar que el parque está cerrado en el
día y horario seleccionado")

# Print si el caso de prueba pasa
print("La prueba para validar que se inscriba en una actividad
indicando una fecha donde el parque se encuentra cerrado pasa con éxito")
```

```
def test_inscribirse_a_actividad_con_fecha_pasada_fail():
# Precondiciones
tipos_de_actividades = ['Safari', 'Palestra', 'Jardinería', 'Tirolesa']
actividades_a_realizar = [
'actividad':{
'tipo': 'Safari',
'duracion': 30,
'horarios': ['10:00', '11:00', '12:00']
'cupos_total': 15,
'cupos_restantes_por_fecha': [
{
'fecha': '01-10-2025',
'cupos': [
{'horario': '10:00', 'cupos_restante': 0},
{'horario': '11:00', 'cupos_restante': 1},
{'horario': '12:00', 'cupos_restante': 14},
]
},
{
'fecha': '02-10-2025',
'cupos': [
{'horario': '10:00', 'cupos_restante': 15},
{'horario': '11:00', 'cupos_restante': 15},
{'horario': '12:00', 'cupos_restante': 15},
]
}
],
'fechas_actividad': ['01-10-2025', '02-10-2025', '03-10-2025'],
'requiere_vestimenta': false
},
'actividad'{
'tipo': 'Palestra',
'duracion': '45',
'horarios': ['10:00', '11:00', '12:00']
'cupos': 20,
```

```
        'cupos_restantes': [20, 19, 14],
        'fechas_actividad': ['01-10-2025', '02-10-2025', '03-10-2025'],
        'requiere_vestimenta': true
    }
]
# dias_no_disponibles = ['29-09-2025', '30-09-2025', '5-10-2025', '6-
10-2025']
inscripcion = {
    "horario": "10/10/2004:18:30:00",
    "cantidad": "1",
    "actividad": "Tirolesa",
    "participantes": [
        {'nombre': 'Ezequiel Gustavo Izaguirre',
         'dni': '43997008',
         'edad': '23',
         'talla_vestimenta': ""}],
    "terminos": "True"}

# Secuencia
seleccionar_boton('Inscribirme a actividad')

# Resultado
try:
    resultado_esperado =inscribirse_a_actividad(inscripcion,
actividades_a_realizar)
except (ExceptionType FechaPasada ex):
    assertEquals(ex["message"] == "No se puede inscribir a una
actividad indicando una fecha en el pasado", "No se informa el ingreso de
una fecha pasada")

# Print si el caso de prueba pasa
print("La prueba para validar que se inscriba en una actividad sin
cupos disponible pasa")
```

```
def test_inscribirse_a_actividad_success():
    # Precondiciones
    tipos_de_actividades = ['Safari', 'Palestra', 'Jardinería', 'Tirolesa']
    actividades_a_realizar = [
        'actividad':{
            'tipo': 'Safari',
            'duracion': 30,
            'horarios': ['10:00', '11:00', '12:00']
            'cupo_total': 15,
            'cupos_restantes_por_fecha': [
                {
```



```
        'fecha': '01-10-2025',
        'cupos': [
            {'horario': '10:00', 'cupo_restante': 0},
            {'horario': '11:00', 'cupo_restante': 1},
            {'horario': '12:00', 'cupo_restante': 14},
        ]
    },
    {
        'fecha': '02-10-2025',
        'cupos': [
            {'horario': '10:00', 'cupo_restante': 15},
            {'horario': '11:00', 'cupo_restante': 15},
            {'horario': '12:00', 'cupo_restante': 15},
        ]
    }
],
'fechas_actividad': ['01-10-2025', '02-10-2025', '03-10-2025'],
'requiere_vestimenta': false
},
'actividad'{
    'tipo': 'Palestra',
    'duracion': '45',
    'horarios': ['10:00', '11:00', '12:00']
    'cupo': 20,
    'cupos_restantes': [20, 19, 14],
    'fechas_actividad': ['01-10-2025', '02-10-2025', '03-10-2025'],
    'requiere_vestimenta': true
}
]
# dias_no_disponibles = ['29-09-2025', '30-09-2025', '5-10-2025', '6-
10-2025']
inscripcion = {
    "horario": "1/10/2025:12:00:00",
    "cantidad": "1",
    "actividad": "Safari",
    "participantes": [
        {'nombre': 'Ezequiel Gustavo Izaguirre',
        'dni': '43997008',
        'edad': '23',
        'talla_vestimenta': ""}],
    "terminos": "True"}

# Secuencia
seleccionar_boton('Inscribirme a actividad')
resultado_esperado = inscribirse_a_actividad(inscripcion,
actividades_a_realizar)
# Resultado
assert (resultado_esperado.objetType == Inscripcion, "Se debe crear un
```

```
objeto de tipo Inscripcion al inscribirse a una actividad")
    assertEquals(resultado_esperado.horario == "1/10/2025:12:00:00", "El
horario de la inscripción creada no coincide con el ingresado")
    assertEquals(resultado_esperado.cantidad == 1, "La cantidad de la
inscripción creada no coincide con la ingresada")
    assertEquals(resultado_esperado.actividad == "Safari", "La actividad de
la inscripción creada no coincide con la ingresada")
    assertEquals(participanteInscripto ==
inscripcion["participantes"]["nombre"] for participanteInscripto in
inscripcion["participantes"], "No todos los participantes inscriptos
coinciden con los ingresados")
    assertEquals(resultado_esperado.terminos == True, "La aceptación de los
Terminos y Condiciones de la inscripción creada no coinciden con la
indicada")

# Print si el caso de prueba pasa
print("La prueba para validar que se inscriba en una actividad pasa")
```

Pseudocódigo función principal y auxiliares:

```
def inscribirse_a_actividad(inscripcion, actividades_a_realizar):
    # Validar Terminos y condiciones
    if not verificar_terminos(inscripcion["terminos"]):
        return "No se han aceptado los términos y condiciones.
Por favor, acéptelos para poder inscribirse a la actividad"

    # Validar talla requerida
    if not validar_talla_actividad(inscripcion, actividades_a_realizar):
        return "No se indicó el talla requerido para poder realizar la
actividad"

    #Validar DNI ingresado
    if not validar_DNI_participante(inscripcion):
        throw Exception DocumentoInvalido("DNI en formato no numérico")

    #Validar parque abierto:
    if not validar_parque_abierto(inscripcion["horario"].day,
dias_cerrado):
        return "El parque se encuentra cerrado en el día y horario
seleccionado"

    #Validar inscripcion a fecha pasada
    if not validar_fecha_pasada(inscripcion["horario"]):
        throw Exception FechaPasada("No se puede inscribir a una
actividad indicando una fecha en el pasado")

    #Validar cupo
```

```
        if not validar_cupo_actividad(inscripcion["horario"].date,
inscripcion["actividad"], inscripcion["horario"].time,
actividades_a_realizar):
            return "No hay cupos para ese dia y horario seleccionado"

# Confirmar inscripción
inscripcion = new Inscripcion(inscripcion)
return inscripcion
```

```
def validar_cupo_actividad(fecha, tipo_actividad_seleccionada,
horario_seleccionado, actividades, dias_cerrado):
    for actividad in actividades:
        if actividad[tipo] == tipo_actividad_seleccionada:
            for dia in actividad[cupos_restantes_por_fecha]:
                if dia[fecha] == fecha:
                    for cupo in dia[cupos]:
                        if cupo[horario] == horario_seleccionado:
                            return cupo[cupo_restante] > 0

    return false
```

```
def validar_talla_actividad(inscripcion, actividades_a_realizar):
    actividad = actividades_a_realizar.filter(act -> act["tipo"] ==
inscripcion["actividad_seleccionada"])
    if actividad.requiere_talla():
        for persona in inscripcion.participantes:
            if not persona.talla:
                return False

    return True
```

```
def validar_DNI_participante(inscripcion):
    if (not isinstance(inscripcion["participantes["DNI"]"], Integer)) or
(len(inscripcion["participantes["DNI"]"]) > 8):
        return False
    else:
        return True
```

```
def validar_parque_abierto(fecha, dias_cerrado):
    return not fecha in dias_cerrado
```

```
def validar_fecha_pasada(fecha):
    return fecha < sys.dateTime()
```

PRÁCTICO 7 - SCRUM - Ejercicio de aplicación (Evaluable)

Unidad:	Unidad Nro. 2: Gestión Lean-Ágil de Productos de Software
Consigna:	Por medio de las dinámicas propuestas por los docentes internalizar los conceptos del framework SCRUM.
Objetivo:	Profundizar el conocimiento del grupo de estudiantes en los conceptos, prácticas y lineamientos definidos en el framework SCRUM a través de la ejecución de un caso práctico.
Propósito:	Familiarizarse con la forma gestionar el trabajo de un equipo, utilizando el framework de Scrum, empleando para ello dinámicas colaborativas.
Entradas:	● Guía Oficial de Scrum 2020 ● Conceptos teóricos sobre el tema, desarrollados en clase. ● Materiales necesarios, que deberán ser provistos por los estudiantes. ● Estimación de las User Stories, detallando las actividades necesarias para cada una
Salida:	Cada equipo de estudiantes con conocimientos esclarecidos relacionados a lo que propone SCRUM. La dinámica de los juegos facilitará el debate sobre los resultados obtenidos y sobre la posición que tiene cada uno respecto a estos temas.
Instrucciones:	A través de una dinámica de juego y de las instrucciones comunicadas por los docentes, implementar las siguientes prácticas de Scrum: ● Priorización y planificación de Sprint, definiendo el objetivo del sprint ● Ejecución de Sprint ● Daily Scrum Meeting ● Sprint Review Meeting ● Sprint Retrospective El docente asumirá el rol de instructor para moderar la ejecución de cada dinámica.

Proyecto: Parque de diversiones

Objetivo: Construir prototipos de atracciones de parques de diversiones en miniatura para una reconocida empresa de entretenimiento en Córdoba. La compañía desea adquirir modelos innovadores de atracciones infantiles que puedan servir como inspiración para futuros parques y juguetes interactivos. Para ello, ha solicitado la creación de prototipos que permitan evaluar las mejores propuestas en términos de creatividad, seguridad y diversión.

Materiales que cada grupo deberá tener para el desarrollo de la clase:

- 1 caja vacía y pequeña (tamaño deseado, para la casa)
- 1 caja vacía de zapatos o mas grande (Para la pista de autos)
- 2 palitos de madera (palitos de helado)
- Cartulinas
- Hilo encerado - choricero
- Conos de papel higiénico o papel de cocina
- 8 sorbetes o pajitas
- Tijeras - Marcadores de colores - Cinta adhesiva - Goma de pegar

Reglas:

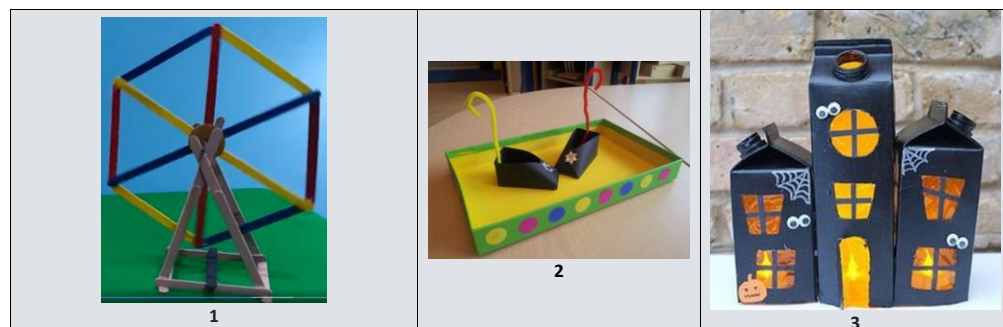
- Al final de cada sprint, se evaluará cuántas atracciones fueron construidas y si cumplen con los requisitos establecidos. Solo se aceptarán aquellas que pasen la revisión de calidad y seguridad. Las atracciones sin terminar podrán continuar en la siguiente iteración hasta su finalización.
- Cada equipo representará una empresa de diseño de parques y tendrá un logotipo con su número de grupo.

Tema: Como empresa de entretenimiento, deseamos ofrecer la mejor experiencia en parques de diversiones y convertirnos en el destino preferido de las familias en Córdoba..

Epic: Contar con una variedad de atracciones innovadoras para niños.

ID	User Stories	Valor de Negocio
1	Yo como dueño del parque quiero contar con una rueda de la fortuna en miniatura, a fin de incluir una atracción clásica para todas las edades.	30
2	Yo como dueño del parque quiero contar con una pista de autos chocadores en miniatura, a fin de incluir una atracción divertida e interactiva.	10
3	Yo como dueño del parque quiero contar con una casa del terror en miniatura, a fin de ofrecer una experiencia temática que atraiga visitantes.	10
4	Yo como dueño del parque quiero contar con una montaña rusa en miniatura, a fin de ofrecer la atracción más emocionante para los niños.	25
5	Yo como dueño del parque quiero contar con dos hamacas en miniatura, a fin de poseer un juego para los más pequeños.	15
6	Yo como dueño del parque quiero contar con una calesita, a fin de incluir una atracción para niños pequeños	10

Imágenes (a modo ilustrativo) de los prototipos deseados





4



5



6

Forma de trabajo / Reglas:

- Cantidad de sprint: 3
 - Tiempos dentro del sprint:
 - Priorización y planificación definiendo el objetivo del sprint: 2 min
 - Ejecución del sprint: 12 min
 - Daily scrum meeting: incluido en sprint, 3. El sprint se divide en 3 días. Cada 4 min deben realizar una.
 - Sprint Review meeting: 4 min
 - Sprint retrospective: 2 min
- Preguntas a realizar:
 - Feedback
 - ¿Realizaron planning?
 - ¿Qué hicieron en la daily?
 - ¿Qué aprendieron en la retro?
- Se puede solicitar cambios durante la ejecución del sprint, en alguna atracción para explicar que durante el sprint no se cambian los requerimientos.
 - Ejemplo: Pedir agregar una flor en la rueda de la fortuna
- Se puede pedir nombre y logo de parque por grupo.

PRÁCTICO 8 - TESTING - Métodos de Caja Negra

Unidad:	Unidad Nro. 4: Aseguramiento de Calidad de Proceso y de Producto																		
Consigna:	Realice el diseño de las pruebas de caja negra según los métodos de Clases de Equivalencia y Análisis de Valor Límite																		
Objetivo:	Que el estudiante pueda diseñar pruebas mediante los métodos de Clases de Equivalencia y Análisis de Valor Límite, para historias de usuario dadas.																		
Propósito:	Familiarizarse con los métodos de Clases de Equivalencia y Análisis de Valor Límite para el diseño de pruebas de caja negra. Descubrir escenarios de prueba que no se derivan a través de los métodos.																		
Entradas:	Conceptos teóricos sobre el tema, desarrollados en clase. Bibliografía referenciada sobre el tema. Enunciado, consigna y User Story.																		
Salida:	Se realizará una puesta en común del conjunto de clases de equivalencia y el resultado de la aplicación del método de análisis del valor límite.																		
	Este práctico no se entrega y por lo tanto no tiene nota. El tema se evalúa en el parcial.																		
Instrucciones:	Según el método de partición de equivalencia, defina las clases existentes utilizando el siguiente cuadro para la Historia de Usuario dada.																		
	<table border="1"> <thead> <tr> <th>Condición externa</th><th colspan="2">Clases de equivalencia válidas</th><th colspan="2">Clases de equivalencia inválidas</th></tr> </thead> <tbody> <tr> <td></td><td></td><td></td><td></td><td></td></tr> <tr> <td></td><td></td><td></td><td></td><td></td></tr> </tbody> </table>				Condición externa	Clases de equivalencia válidas		Clases de equivalencia inválidas											
Condición externa	Clases de equivalencia válidas		Clases de equivalencia inválidas																
	Describa la cantidad mínima de casos de prueba que requiera para asegurar la prueba de todas las clases identificadas y para asegurar que se realicen todas las pruebas de aceptación requeridas por el PO, aplicando los métodos de <i>Partición de Equivalencias</i> . Utilice el template Template_Casos_De_Prueba.																		

Taxi Mobile: Sistema web mobile para seguimiento de taxis

Objetivo: Desarrollar un sistema para celulares para que los pasajeros de taxis puedan solicitar el taxi más cercano y saber su ubicación y demora en todo momento.

A continuación, se transcribe parte de la entrevista realizada al experto en el dominio:

Product Owner (PO): ¿La idea es que el pasajero pueda llamar a una central sabiendo dónde está el taxi que va a pedir? ¿Cómo pide un taxi el pasajero?

Experto en el Dominio (ED): El pasajero debe ingresar a la aplicación, la cual activará el sistema de posicionamiento. El sistema detecta dónde está el pasajero y le muestra los 5 taxis más próximos a su ubicación, visualizados en un mapa, e informa ubicación, distancia y tiempo estimado. El pasajero selecciona el taxi deseado y esta acción envía una notificación a la central de taxis y al taxista a su celular. De esta forma ambos identifican el pedido de un móvil, y el taxista puede saber a dónde está el pasajero

PO: ¿Y cómo sabe el taxista quién es el pasajero?

ED: Cuando se instala la aplicación en el celular se pedirán los datos mínimos para registrarlo como usuario. Estos datos se pueden obtener de Facebook y el número de celular del mismo aparato donde se instala la aplicación.

PO: ¿Y los taxistas cómo trabajan con esta aplicación?

ED: Como taxista también es necesario instalar una aplicación en su celular. El taxista debe estar registrado en una central que haya contratado el servicio de taxi-mobile. Cuando instale la aplicación se solicitarán sus datos identificatorios como taxista y

de su móvil (nro. de chapa), y la selección de la central a la que pertenece. La aplicación también utiliza el sistema de posicionamiento que tiene el teléfono, para poder informar en qué lugar se encuentra el taxista.

PO: ¿Si estoy entendiendo bien, tanto el pasajero como el taxista deben contar con un Smart Phone con sistema de posicionamiento para que la aplicación funcione?

ED: Así es, sino la aplicación no puede ubicar al taxi ni al pasajero.

PO: ¿Y una vez que el pasajero sube al taxi, ¿cómo se indica que ya está en viaje?

ED: El taxista debe asentar esto en la aplicación con la opción correspondiente. De esta forma la central está al tanto de la situación. Igualmente, cuando el pasajero descienda el taxista debe indicarlo, para que el taxi quede libre en la aplicación, y se muestre a un próximo pasajero.

PO: ¿La aplicación sólo muestra taxis libres?

ED: A los pasajeros sí, en la central se pueden ver con distintos colores los taxis ocupados, los libres, los solicitados y los que están fuera de servicio. Pasando el mouse sobre el ícono del auto deben poder visualizarse los datos del viaje si el taxi está ocupado (hora de inicio y costo) y los datos del pasajero (nombre, apellido y número de teléfono celular). Si el taxi está solicitado, sólo se visualizarán los datos del pasajero.

PO: ¿Y la aplicación para la central también es mobile?

ED: No, debería ser una aplicación web. Y podrán utilizarse filtros por Barrio, por Estado y por chapa del taxi para visualizarlos en el mapa.

PO: ¿La idea es que la aplicación del pasajero y del taxista sean con touch screen?

ED: Los elementos centrales de la aplicación son el mapa y el taxi. Siempre a través de la selección del taxi se indicarán las acciones que quieren realizarse, como llamarlo, indicar que está ocupado, que está libre, tanto por parte del pasajero como del taxista.

Ver mapa de taxis	5
Como Administrador de la Central quiero ver la ubicación de todos los taxis de la central y si tienen viajes en curso para saber la disponibilidad actual Criterios de Aceptación: ● Se muestran taxis en todos los estados (Verde: Libre, Amarillo: Solicitado, Rojo: Ocupado, Negro: Fuera de Servicio). ● Los taxis que se mostrarán deben tener habilitado el sistema de ubicación. ● Pasando el mouse sobre el ícono del auto deben poder visualizarse los datos del viaje si el taxi está ocupado (hora de inicio y costo) y los datos del pasajero (nombre, apellido y número de teléfono celular). ● Si el taxi está solicitado, sólo se visualizarán los datos del pasajero. ● Podrán utilizarse filtros por Barrio, por Estado y por chapa del taxi Pruebas de Usuario ● Probar visualizar un conjunto de taxis libres (pasa) ● Probar visualizar un conjunto de taxis solicitados (pasa) ● Probar visualizar un conjunto de taxis fuera de servicio (pasa) ● Probar visualizar un conjunto de taxis ocupados (pasa) ● Probar visualizar un conjunto de taxis libres, solicitados, fuera de servicio y ocupados (pasa) ● Probar visualizar un taxi conectado al sistema por número de chapa (pasa) ● Probar visualizar en el mapa un taxi no conectado al sistema de ubicación (falla) ● Probar visualizar un conjunto de taxis en un Barrio inexistente (falla)	

SOLUCIÓN PRÁCTICO 9 - TESTING

Condición externa	Clases de equivalencia válidas		Clases de equivalencia inválidas	
Barrio	1	Barrio existente	2	Barrio inexistente
Estado del taxi	3	Libre	7	Cualquier estado distinto de los válidos
		Solicitado		
		Ocupado		
		Fuera de servicio		
Chapa del taxi	8	Matrícula de taxi (3 letras + 3 números)	9	Cualquier otra combinación de números y letras
		Matrícula de taxi (2 letras + 3 números + 2 letras)		
Usuario logueado	10	Usuario con perfil de Administrador de la Central	12	Usuario no logueado
				Cualquier usuario con perfil distinto a Administrador de la Central
Sistema ubicación		Sistema de ubicación habilitado		Sistema de ubicación no habilitado
Clases de Equivalencia de Salida				
Identificador del estado del taxi	13	Verde, Amarillo, Rojo, Negro	17	"Taxi no conectado al Sistema de Ubicación"
Identificador del pasajero	18	Nombre y Apellido, Celular	19	"Error, no es posible mostrar los datos del pasajero"
Hora inicio del viaje	20	hh:mm dd/mm/yyyy correspondiente a una fecha – hora menor o igual a la actual	21	"Error, no es posible mostrar el inicio del viaje"
Costo del viaje	22	Precio decimal mayor a cero	23	"Error, no es posible mostrar el costo del viaje"
Identificador del taxi	24	Matrícula del Taxi	25	"Taxi no conectado al Sistema de Ubicación"

Condición externa	Clases de equivalencia válidas		Clases de equivalencia inválidas	
Mensaje de búsqueda de taxi	26		27	“Barrio Inexistente”

Id	Prioridad	Nombre del Caso de Prueba	Precondiciones	Pasos	Resultado esperado
1	Alta	Taxis Libres para un Barrio	El usuario “pedro” se encuentra logueado y tiene permisos de Administrador de la Central. El software está configurado en el radio de la Ciudad de Córdoba. Los barrios donde la empresa de taxis presta servicios se encuentran cargados en el sistema. Hay taxis conectados al sistema de ubicación y en estado “Libre” en la ciudad “Córdoba”, en el barrio “Nueva Córdoba”	1. El Administrador de la Central selecciona la opción “Ver mapa de taxis” 2. El Administrador de la Central ingresa el barrio “Nueva Córdoba” 3. El Administrador de la Central ingresa estado “Libre” 4. El Administrador de la Central selecciona la opción “Mostrar mapa”	- El sistema muestra el mapa de la ciudad de Córdoba, haciendo zoom en el barrio de Nueva Córdoba, con los taxis en estado Libre. - Los taxis se identifican en el mapa con el ícono de un auto color Verde.
2	Alta	Taxis Solicitados para un Barrio	El usuario “pedro” se encuentra logueado y tiene permisos de Administrador de la Central.	1. El Administrador de la Central selecciona la opción “Ver mapa de taxis” 2. El Administrador de la Central ingresa el barrio “Nueva Córdoba” 3. El Administrador de la Central ingresa estado “Solicitado” 4. El Administrador de la Central selecciona la opción “Mostrar mapa”	- El sistema muestra el mapa de la ciudad de Córdoba, haciendo zoom en el barrio de Nueva Córdoba, con los taxis en estado Solicitado. - Los taxis se identifican en el mapa

Id	Prioridad	Nombre del Caso de Prueba	Precondiciones	Pasos	Resultado esperado
			El software está configurado en el radio de la Ciudad de Córdoba Los barrios donde la empresa de taxis presta servicios se encuentran cargados en el sistema. Hay taxis conectados al sistema de ubicación y en estado “Solicitado” en la ciudad “Córdoba”, en el barrio “Nueva Córdoba” El taxi IXX 685 ha sido “Solicitado” por el pasajero: “Marisa Peralta, cel 3513455999”		con el ícono de un auto color Amarillo. - Para el taxi IXX 685 se muestran los datos del pasajero: “Marisa Peralta, cel 3513455999”
3	Alta	Taxis Ocupados para un Barrio	El usuario “pedro” se encuentra logueado y tiene permisos de Administrador de la Central. El software está configurado en el radio de la Ciudad de Córdoba Los barrios donde la empresa de taxis presta servicios se encuentran	1. El Administrador de la Central selecciona la opción “Ver mapa de taxis” 2. El Administrador de la Central ingresa el barrio “Nueva Córdoba” 3. El Administrador de la Central ingresa estado “Ocupado” 4. El Administrador de la Central selecciona la opción “Mostrar mapa”	- El sistema muestra el mapa de la ciudad de Córdoba, haciendo zoom en el barrio de Nueva Córdoba, con los taxis en estado Ocupado. - Los taxis se identifican en el mapa con el ícono de un auto color Rojo. - Para el taxi IXT 689 se muestra el ícono en color Rojo y al pasar el mouse sobre el ícono se muestran los datos del viaje: “14/10 16:10 horas, costo \$48” y los datos del pasajero:

Id	Prioridad	Nombre del Caso de Prueba	Precondiciones	Pasos	Resultado esperado
			cargados en el sistema. Hay taxis conectados al sistema de ubicación y en estado “Ocupado” en la ciudad “Córdoba”, en el barrio “Nueva Córdoba” El taxi IXT 689 está en estado “Ocupado” en el barrio de “Nueva Córdoba” los datos del viaje son: “14/10 16:10 horas, costo \$48” y los datos del pasajero: “Mariana Perez, cel 3513455777”		“Mariana Perez, cel 3513455777”
4	Alta	Taxis Fuera de Servicio para un Barrio	El usuario “pedro” se encuentra logueado y tiene permisos de Administrador de la Central. El software está configurado en el radio de la Ciudad de Córdoba Los barrios donde la empresa de taxis presta servicios se encuentran cargados en el sistema.	1. El Administrador de la Central selecciona la opción “Ver mapa de taxis” 2. El Administrador de la Central ingresa el barrio “Nueva Córdoba” 3. El Administrador de la Central ingresa estado “Fuera de Servicio” 4. El Administrador de la Central selecciona la opción “Mostrar mapa”	- El sistema muestra el mapa de la ciudad de Córdoba, haciendo zoom en el barrio de Nueva Córdoba, con los taxis en estado Fuera de Servicio. Los taxis se identifican en el mapa con el ícono de un auto color Negro.

Id	Prioridad	Nombre del Caso de Prueba	Precondiciones	Pasos	Resultado esperado
			Hay taxis conectados al sistema de ubicación y en estado “Fuera de Servicio” en la ciudad “Córdoba”, en el barrio “Nueva Córdoba”		
5	Alta	Conjunto de taxis para un Barrio en todos los estados	El usuario “pedro” se encuentra logueado y tiene permisos de Administrador de la Central. El software está configurado en el radio de la Ciudad de Córdoba Los barrios donde la empresa de taxis presta servicios se encuentran cargados en el sistema. Hay taxis conectados al sistema de ubicación en la ciudad “Córdoba” en el barrio “Nueva Córdoba” Hay al menos un taxi en estado “Libre”, un taxi en estado “Solicitado”, un taxi en estado	1. El Administrador de la Central selecciona la opción “Ver mapa de taxis” 2. El Administrador de la Central ingresa el barrio “Nueva Córdoba” 3. El Administrador de la Central selecciona la opción “Mostrar mapa”	- El sistema muestra el mapa de la ciudad de Córdoba, haciendo zoom en el barrio de Nueva Córdoba, con los taxis conectados al sistema de ubicación. - Los taxis se identifican en el mapa con el ícono de un auto con distintos colores según su estado: Verde: Libre, Amarillo: Solicitado, Rojo: Ocupado, Negro: Fuera de Servicio. - Para el taxi IIT 889 se muestra el ícono en color Rojo y al pasar el mouse sobre el ícono se muestran los datos del viaje: “12/10 15:10 horas, costo \$67” y los datos del pasajero: “Esteban Gomez, cel 3513455444” - Para el taxi ILL 554 se muestra el ícono en color Amarillo y al pasar el mouse sobre el ícono se muestran los datos del pasajero: “Juan Figueroa, cel 3513455222”

Id	Prioridad	Nombre del Caso de Prueba	Precondiciones	Pasos	Resultado esperado
			“Ocupado” y un taxi en estado “Fuera de Servicio” en el barrio “Nueva Córdoba” El taxi IIT 889 está en estado “Ocupado” en el barrio de “Nueva Córdoba” los datos del viaje son: “12/10 15:10 horas, costo \$67” y los datos del pasajero: “Esteban Gomez, cel 3513455444” El taxi ILL 554 ha sido solicitado por el pasajero: “Juan Figueroa, cel 3513455222”		
6	Alta	Visualizar taxi por número de chapa	El usuario “pedro” se encuentra logueado y tiene permisos de Administrador de la Central. El software está configurado en el radio de la Ciudad de Córdoba Los barrios donde la empresa de taxis presta servicios se encuentran cargados en el sistema.	1. El Administrador de la Central selecciona la opción “Ver mapa de taxis” 2. El Administrador de la Central ingresa el número de chapa “IPV 777” 3. El Administrador de la Central selecciona la opción “Mostrar mapa”	- El sistema muestra el mapa de la ciudad de Córdoba, haciendo zoom en el barrio de Nueva Córdoba, mostrando el ícono de un auto color Negro, resaltando el número de chapa “IPV 777”

Id	Prioridad	Nombre del Caso de Prueba	Precondiciones	Pasos	Resultado esperado
			El taxi IPV 777 está conectado al sistema de ubicación y en estado “Fuera de Servicio” en la ciudad “Córdoba”, en el barrio “Nueva Córdoba”		
7	Media	Visualizar taxi no conectado	El usuario “pedro” se encuentra logueado y tiene permisos de Administrador de la Central. El software está configurado en el radio de la Ciudad de Córdoba Los barrios donde la empresa de taxis presta servicios se encuentran cargados en el sistema. El taxi IPV 778 NO está conectado al sistema de ubicación y en estado “Fuera de Servicio” en la ciudad “Córdoba”, en el barrio “Nueva Córdoba”	1. El Administrador de la Central selecciona la opción “Ver mapa de taxis” 2. El Administrador de la Central ingresa el número de chapa “IPV 778” 3. El Administrador de la Central selecciona la opción “Mostrar mapa”	- El sistema muestra el mensaje “Taxi No Conectado al Sistema de Ubicación”
8	Baja	Visualizar taxi en	El usuario “pedro” se encuentra logueado y tiene	1. El Administrador de la Central selecciona la opción “Ver mapa de taxis”	- El sistema muestra el mensaje de error “El barrio XXX Córdoba no

Id	Prioridad	Nombre del Caso de Prueba	Precondiciones	Pasos	Resultado esperado
		Barrio inexistente	permisos de Administrador de la Central. El software está configurado en el radio de la Ciudad de Córdoba Los barrios donde la empresa de taxis presta servicios se encuentran cargados en el sistema. El Barrio “XXX Córdoba” no se encuentra cargado en el sistema.	2. El Administrador de la Central ingresa el barrio “XXX Córdoba” 3. El Administrador de la Central selecciona la opción “Mostrar mapa”	se encuentra cargado en el Sistema”

PRÁCTICO 9 - TESTING - Métodos de Caja Negra

Comentado [1]: Falta agregar las clases de comprar entradas

Unidad:	Unidad Nro. 4: Aseguramiento de Calidad de Proceso y de Producto																							
Consigna:	Realice el diseño de las pruebas de caja negra según los métodos de Clases de Equivalencia y Análisis de Valor límite																							
Objetivo:	Que el estudiante pueda diseñar pruebas mediante los métodos de Clases de Equivalencia y Análisis de Valor límite, para historias de usuario dadas.																							
Propósito:	Familiarizarse con los métodos de Clases de Equivalencia y Análisis de Valor límite para el diseño de pruebas de caja negra. Descubrir escenarios de prueba que no se derivan a través de los métodos.																							
Entradas:	Conceptos teóricos sobre el tema, desarrollados en clase. Bibliografía referenciada sobre el tema. Enunciado, consigna y User Story. Planilla de Excel con Plantilla de Casos de Prueba y Reporte de Defectos Deliverat_Template_Caso_De_Prueba_Defectos.xlsx																							
Salida:	Se realizará una puesta en común del conjunto de clases de equivalencia y el resultado de la aplicación del método de análisis del valor límite. Este práctico no se entrega y por lo tanto no tiene nota. El tema se evalúa en el parcial.																							
Instrucciones:	Según el método de partición de equivalencia, defina las clases existentes utilizando el siguiente cuadro para la User Story de las siguientes que no haya implementado en el Trabajo Práctico 6 “Requerimientos Ágiles – Implementación de User Stories”, sobre EcoHarmony Park :																							
	○ Comprar entradas (grupos impares) ○ Inscribirse a actividad (grupos pares)																							
	<table><thead><tr><th>Condición externa</th><th colspan="2">Clases de equivalencia válidas</th><th colspan="2">Clases de equivalencia inválidas</th></tr></thead><tbody><tr><td></td><td></td><td></td><td></td><td></td></tr><tr><td></td><td></td><td></td><td></td><td></td></tr><tr><td></td><td></td><td></td><td></td><td></td></tr></tbody></table>				Condición externa	Clases de equivalencia válidas		Clases de equivalencia inválidas																
	Condición externa	Clases de equivalencia válidas		Clases de equivalencia inválidas																				
Describa la cantidad mínima de casos de prueba que requiera para asegurar la prueba de todas las clases identificadas y para asegurar que se realicen todas las pruebas de aceptación requeridas por el PO, aplicando los métodos de <i>Partición de Equivalencias</i> y de <i>Análisis de Valores Límite</i> (para las condiciones externas en las que sea posible).																								
Utilice el template <i>Template Casos De Prueba</i> .																								

SOLUCIÓN PRÁCTICO 10 - TESTING

Condición externa	Clases de equivalencia válidas	Clases de equivalencia inválidas
CLASES DE EQUIVALENCIA DE ENTRADA		
Actividad	1 Seleccionar una actividad con cupo de la lista: "Tirolesa", "Safari", "Palestra" y "Jardinería" con	2 No selecciona actividad
		3 Selecciona una actividad sin cupo
Fecha y Horario	4 Selecciona una fecha en formato dd/mm/aaaa y horario disponible en formato hh:00, donde fecha actual <= fecha seleccionada <= fecha actual + 30 días. La fecha seleccionada no debe ser lunes o día festivo. El horario disponible debe ser entre 9 a 18h.	5 No selecciona fecha y hora
		6 Selecciona un horario antes de las 9
		7 Selecciona un horario anterior a la hora actual
		8 Selecciona un horario después de las 18
		9 Selecciona fecha anterior a la actual
		10 Selecciona fecha posterior a un mes
		11 Selecciona fecha lunes
		12 Selecciona fecha día festivo
Cantidad de personas	13 1 <= cantidad de personas <= 8 para actividad "Safari" y menor o igual al cupo disponible	17 Cantidad de personas <= 1
	14 1 <= cantidad de personas <= 12 para actividad "Palestra" y menor o igual al cupo disponible	18 No ingresa cantidad de personas
	15 1 <= cantidad de personas <= 12 para actividad "Jardinería" y menor o igual al cupo disponible	19 Ingresa carácter no numérico entero
	16 1 <= cantidad de personas <= 10 para actividad "Tirolesa" y menor o igual al cupo disponible	20 Numero mayor al cupo disponible
		21 Cantidad de personas >= 8 para "Safari"
		22 Cantidad de personas >= 12 para "Jardinería"

			23	Cantidad de personas ≥ 12 para "Palestra"
			24	Cantidad de personas ≥ 10 para "Tirolesa"
			25	Cupo de actividad completa
Nombre	26	Ingresa cadena de letras de no más de 50 caracteres	27	No ingresa nombre
			28	Ingresa una cadena que tiene otros caracteres además de letras
			29	Ingresa una cadena de más de 50 caracteres
DNI	30	Ingresa dígitos correspondientes al formato del DNI	31	No ingresa DNI
			32	Ingresa una cadena no numérica
			33	Ingresa dígitos que no corresponde al formato de DNI (9999999 o 99999999)
Talla de vestimenta	34	Selecciona talla entre XS al XL ya que la actividad lo requiere	36	No selecciona talla de vestimenta y la actividad lo requiere
	35	No selecciona talla y la actividad no lo requiere		
Edad	37	Ingresa edad: $0 \leq \text{edad} \leq 110$ años en actividad "Jardinería" en formato número entero	41	No ingresa edad
	38	Ingresa edad: $0 \leq \text{edad} \leq 110$ años en actividad "Safari" en formato número entero	42	Ingresa edad en formato no numérico entero
	39	Ingresa edad: $12 \leq \text{edad} \leq 110$ años en actividad "Palestra" en formato número entero	43	Ingresa edad < 0
	40	Ingresa edad: $8 \leq \text{edad} \leq 110$ en actividad "Tirolesa" en formato número entero	44	Ingresa edad > 110
			45	Ingresa edad < 12 en actividad "Palestra"
			46	Ingresa edad < 8 en actividad "Tirolesa"
Términos condiciones	y 47	Acepta los términos y condiciones	48	No acepta los términos y condiciones
Usuario	49	Usuario logueado y con permisos de visitante	50	Usuario no logueado

			51	Usuario sin permisos de visitante
CLASES DE EQUIVALENCIA DE SALIDA				
Mensaje de confirmación de inscripción	52	Mensaje de confirmación a inscripción indicando fecha, horario y nombre de la actividad	54	Mensaje de error informando “No se ha seleccionado la actividad”
			55	Mensaje de error informando “La actividad seleccionada no tiene cupo”
			56	Mensaje de error informando “No se ha seleccionado un horario”
			57	Mensaje de error informando “Se seleccionó un horario por fuera del rango permitido”
			58	Mensaje de error informando “No se ha ingresado nombre”
			59	Mensaje de error informando “Debe ingresar una cadena válida”
			60	Mensaje de error informando “Debe ingresar un DNI”
			61	Mensaje de error informando “Debe ingresar una cadena numérica”
			62	Mensaje de error informando “Debe ingresar un DNI correspondiente a una persona”
			63	Mensaje de error informando “Debe seleccionar talla de vestimenta”
			64	Mensaje de error informando “Debe seleccionar rango de edad”
			65	Mensaje de error informando “Se debe aceptar los términos y condiciones”
Mensaje Push al responsable de la actividad	53	Llega un mensaje push al responsable de la actividad informando el total de inscriptos hasta el momento		

PRÁCTICO 10 - KANBAN – Ejercicio de aplicación (Evaluable)

Unidad:	Unidad Nro. 4: Aseguramiento de Calidad de Proceso y de Producto
Consigna:	A través de un juego y de las instrucciones comunicadas por los docentes, vivenciar los principios kanban
Objetivo:	Experimentar cómo un sistema Kanban emerge de un proceso existente, como ocurre en el mundo real. Vivenciar un sistema Kanban completo, en lugar de centrarse sólo en el tablero Kanban y las mecánicas relacionadas. Comprender que los tableros dependen del contexto: para cualquier proceso dado hay muchos diseños de tableros diferentes que son adecuados y útiles, pero el tablero perfecto no necesariamente existe. Comprender los efectos de limitar su trabajo en progreso. Experimentar la autoorganización y la adaptación.
Propósito:	Familiarizarse la filosofía lean, internalizando los principios propuestos por Kanban, utilizando una dinámica grupal, en forma de juego.
Entradas:	• Conceptos teóricos sobre el tema, desarrollados en clase. Bibliografía referenciada sobre el tema. • Instrucciones del juego. • Materiales necesarios que deben ser llevado por los estudiantes.
Salida:	Mostrar los resultados de la dinámica en cada iteración.
Instrucciones:	El docente que dirige la dinámica será quién indique la consigna y el alcance de la dinámica a realizar.
Lista de materiales	• Post-Its en tres colores: amarillo (piña), rosa (jamón) y verde (rúcula) • Papel de impresora para cortar fondos de pizza (A4 / Carta u otros tamaños) • Marcadores rojos como salsa de tomate. • Pegamento o cinta transparente (para hacer que el Post-Its se pegue mejor) • Cinta de enmascarar (también conocida como cinta de pintor) • Tijeras (una pequeña + una grande por equipo) • Cronómetro • Tarjetas de pedido - un juego por equipo • Placa de horno - una por equipo • Hojas de reglas • • Hojas de información general de Kanban

Los principios de Kanban

- **Visualizar el flujo de trabajo:** Normalmente está escondido de alguna manera, se trata de hacerlo explícito y visible por todos.
- **Limitar el WIP:** Limitamos el trabajo el curso. Se limita para evitar cuellos de botella, y para garantizar que solo entran elementos nuevos cuando hay capacidad de completarlos (no se produce por encima del cuello de botella).
- **Gestionar y medir el flujo:** Medir para ver si se mejora o no, y decidir en consecuencia. Podría ser número de tareas completas en el Sprint, o número de Story Points completados, el Lead Time, el tiempo de ciclo, etc...
- **Implementar ciclos de feedback:** En el mundo ágil se corresponden con las retrospectivas. La idea de Kanban va más allá y define que se debe recibir el feedback en todos los niveles (del equipo, del cliente, del PO, stakeholders...)
- **Explicitar políticas y procedimientos:** Por ejemplo los “Definition of Done”, “Definition of Ready”, valores y principios
- **Mejorar continuamente mediante la colaboración:** Usando modelos y experimentando científicamente. Muchas veces es lo que falla de las retrospectivas, se decide algo y después no se sigue ni se mide la consecuencia de aplicarlo. Kanban propone algo más científico, definir una hipótesis, implementar un experimento y medir las consecuencias. Si funciona se sigue y si no comienzas de nuevo el ciclo

Reglas de juego:

Pizza de jamón

- Corte de porción de pizza triangular
- Salsa bien cubierta
- 3 Trozos de jamón
- 3 trozos de queso
- *Todo al horno*

Pizza de rúcula

- Corte de porción de pizza triangular
- Salsa bien cubierta
- 7 Trozos de rúcula
- *La rúcula se agrega luego, no ingresa al horno*

Puntajes:

- Porción de pizza finalizada: 10 puntos
- Porción de pizza sin terminar (la masa de la porción de pizza está): -4 puntos
- Coberturas sueltas (jamón, queso) y cualquier otra cosa suelta: -1 punto

Ciclos:

- Ciclo 1 - Maximo : La mayor cantidad de pizzas posibles
- Ciclo 2 - Maxima cantidad se agrega la rucula (final)
- Ciclo 3 - Maxima cantidad ya organizado
- Ciclo 4 - Sistema de Pedidos

Prácticas Kanban (Repaso):

1. **Visualizar el flujo de trabajo:** Representar de forma clara los pasos por los que pasa el trabajo desde que se inicia hasta que se completa.
2. **Limitar el trabajo en curso (WIP):** Restringir la cantidad de trabajo que puede estar en proceso al mismo tiempo para evitar cuellos de botella y sobrecarga.
3. **Gestionar el flujo:** Supervisar y mejorar continuamente cómo fluye el trabajo a través del sistema (tiempo de ciclo, tiempo de entrega, rendimiento).
4. **Implementar bucles de retroalimentación:** Establecer mecanismos regulares para revisar el trabajo y los procesos (como reuniones diarias y retrospectivas).
5. **Hacer explícitas las políticas del proceso:** Definir claramente las reglas y criterios para que todos sepan cómo funciona el sistema y puedan tomar decisiones informadas.
6. **Mejorar colaborativamente, evolucionar experimentalmente:** Fomentar la mejora continua mediante la colaboración del equipo y el uso de métricas para experimentar con cambios y ajustes.

Principios:

1. Eliminar Desperdicios
2. Amplificar Aprendizaje
3. Embeber la Integridad conceptual
4. Diferir compromisos
5. Dar poder al equipo
6. Ver el todo
7. Entregar lo antes posible

PRÁCTICO 11 - SCRUM - Dinámica de Retrospectiva (Evaluable)

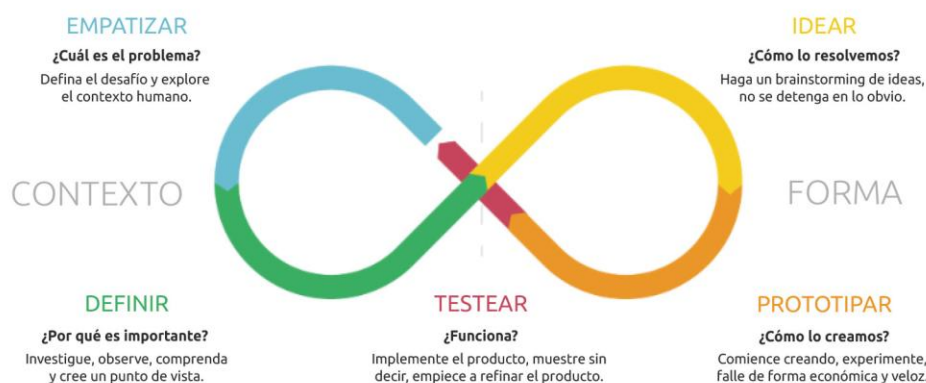
Unidad:	Unidad Nro. 2: Gestión Lean-Ágil de Productos de Software
Consigna:	En función de los conceptos desarrollados en la materia sobre las ceremonias de Scrum y en particular sobre Retrospectivas, aplicar la técnica al trabajo grupal realizado por los estudiantes durante el cursado de la materia.
Objetivo:	Utilizar la técnica de retrospectiva propuesta por SCRUM para realizar una autoevaluación del desempeño de los miembros del grupo de trabajo en el contexto del cursado de esta materia.
Propósito:	Familiarizarse con la gestión ágil de proyectos, conceptos y distintas técnicas de retrospectivas para obtener mejores resultados en los equipos de trabajo SCRUM.
Entradas:	• Conceptos teóricos sobre el tema desarrollados en clase e investigación de diferentes técnicas de retrospectivas de Scrum. • Bibliografía referenciada sobre el tema. • Instrucciones de la dinámica. • Materiales de trabajo (post-its, afiches, fibrones)
Salida:	• Presentación con el resultado de la ejecución de la ceremonia retrospectiva y de la técnica empleada. • Conclusiones obtenidas al finalizar la retrospectiva realizada por cada grupo y las conclusiones generales del grupo clase.
Instrucciones:	• Se trabajará con el grupo de clase. • Investigar y elegir una técnica de retrospectiva. • Realizar la retrospectiva respetando la estructura propuesta: apertura (“icebreaker”), retrospectiva (aplicando técnica elegida), definición de ítems de acción y responsables, conclusiones y cierre. • Presentar la técnica empleada y las conclusiones de cada grupo. • Debatir las conclusiones generales de la ejecución de la ceremonia de retrospectiva con el grupo clase

PRÁCTICO 12 - DESIGN THINKING – Publicidad de Instagram (Evaluable)

Unidad:	Unidad Nro. 2: Gestión Lean-Ágil de Productos de Software
Consigna:	Simular la definición de un producto de software. Para ello se utilizará una dinámica en la que se diseñará la publicidad en Instagram de un producto para vender en el mercado.
Objetivo:	Aplicar algunos conceptos de gestión ágil y design thinking en el desarrollo de productos de software y el concepto de MVP.
Propósito:	Familiarizarse con la gestión ágil de proyectos y algunos conceptos de Design Thinking.
Entradas:	• Conceptos teóricos sobre el tema desarrollados en clase. • Bibliografía referenciada sobre el tema. • Instrucciones de la dinámica. • Materiales de trabajo (a coordinar)
Salida:	• Presentación con el resultado de la ejecución de las Etapas del proceso propuesto. • El diseño de la publicidad de un producto que cumple con el MVP, es decir que es la versión mínima de mi producto que me permite validarlo con el Mercado y obtener feedback del cliente.

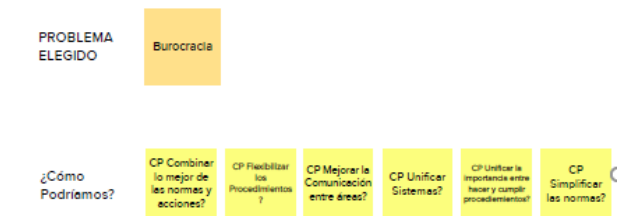
DESIGN THINKING

UN FRAMEWORK PARA INNOVAR



Instrucciones

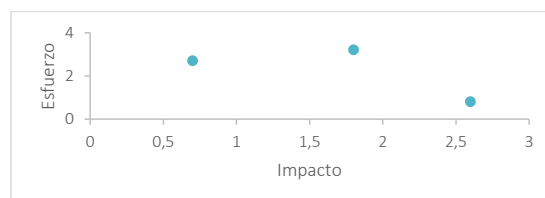
1. Se trabajará sobre un problema que pueda resolverse con un producto de software. Puede ser elegido por cada grupo o planteado por el docente. Por ejemplo, podría plantearse como problema la burocracia en el contexto de la Universidad, el sistema de transporte en Córdoba, los pedidos del Delivery llegan siempre fríos, etc.
2. A partir del problema el grupo planteará los desafíos que el mismo supone, a partir de la consigna "Como Podríamos..." Eso nos permite convertir a los problemas en Desafíos positivos y accionables. Por ejemplo:



3. Y para cada desafío planteamos posibles soluciones

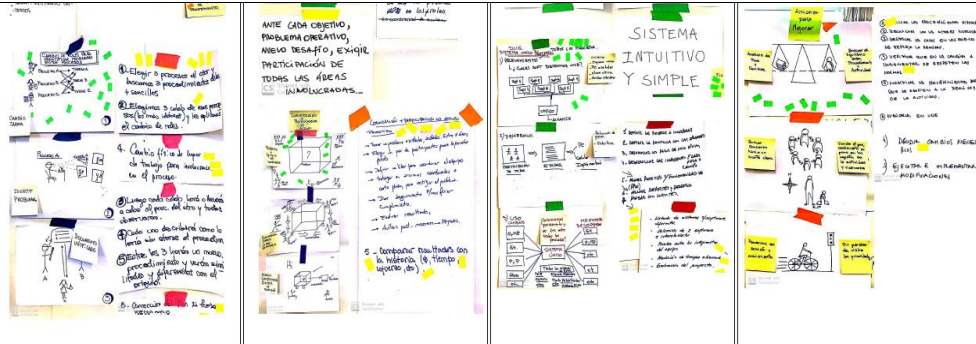


4. Se eligen la solución sobre la cual se trabajará, ponderándolas en una matriz de Esfuerzo – Impacto (se elige la de la combinación de mayor impacto y menos esfuerzo)



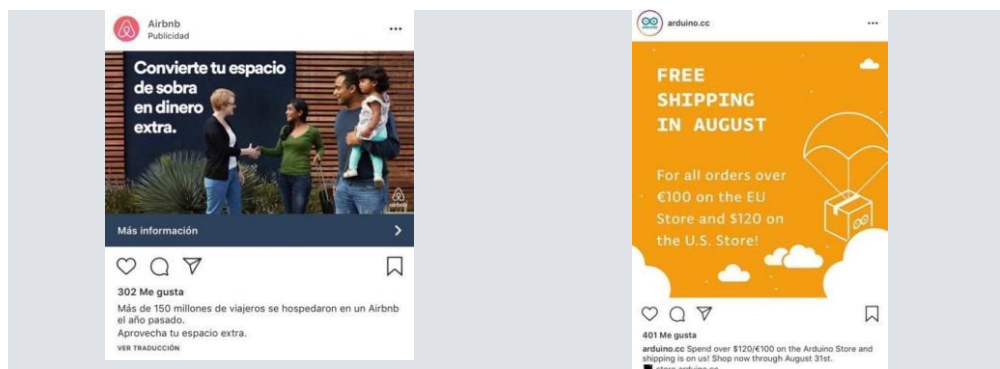
5. Cada alumno del grupo grafica como trabajar sobre la idea elegida en 3 etapas, para modelar el concepto

Ejemplos gráficos:



6. El equipo vota (cada integrante tiene 2 votos) por la representación que más le gustó. La representación más votada es elegida la representación ganadora.
7. El equipo define los requerimientos que debería incluir el Mínimo Producto Viable. Luego arma un prototipo de una publicidad para Instagram de la solución elegida, para el Mínimo Producto Viable.

Ejemplos:



8. El grupo presenta su publicidad a todo el curso.

Time box

- "Como Podríamos" --> 5 minutos
- "Soluciones" --> 5 minutos
- "Esfuerzo/Impacto" --> 5
- "Conceptos/Modelado" --> 15
- "MVP" --> 10
- "Publicidad de ig" --> 15

PRÁCTICO 13 – SCRUM - Planificación de Release y de Sprint (Evaluable) - BONUS

Comentado [2]: Agregar posible resolución

Unidad:	Unidad Nro. 3: Gestión Ágil de Proyectos
Consigna:	Tomando como base la definición de producto realizada para Taxi Mobile: Sistema web mobile para seguimiento de taxis y teniendo en cuenta el MVP definido, realizará con su equipo SCRUM la primera reunión de planificación de Sprint (Sprint Planning).
Objetivo:	Que el estudiante sea capaz de simular una de las ceremonias de SCRUM, Sprint Planning, cuyo propósito es la definición del Sprint Backlog. Que comprenda la importancia de la planificación en el contexto de la gestión ágil de proyectos.
Propósito:	Aplicar los conceptos de Gestión Ágil de Proyectos Vivenciar el ambiente de Scrum simulando la ceremonia de planificación de un sprint.
Entradas:	Conceptos teóricos de SCRUM (actualizados a SGEP 2026). Bibliografía referenciada sobre el tema. Definición de Output Done y Definición de Outcome Done. Caso práctico de Taxi Mobile: Sistema web mobile para seguimiento de taxis previamente desarrollado. Ejemplo de Minuta de Sprint Planning Ejemplo de Sprint Backlog
Salida:	Se evaluará que: ● Presente el plan de release para la liberación de la primera versión del producto (MVP). ● Describa todas las consideraciones de contexto ● Presente la minuta de planificación del Sprint ● El Sprint Backlog desagregando las user stories en tareas estimadas en horas ideales ● Descripción de todas las consideraciones de contexto que considere necesarias para la comprensión de los entregables mencionados en los ítems anteriores.
Instrucciones:	● En grupos trabajarán la consigna. ● Tomarán el MVP definido para el producto de Taxi Mobile: Sistema web mobile para seguimiento de taxis, ya estimado ● Definirán las condiciones de contexto necesarias para la planificación del release. ● Definirán el Plan de Release indicando cuantos sprints serán necesarios y que user stories entregarán en cada uso y por consiguiente la duración del Plan de Release para la entrega de la versión del producto. ● Definirá la minuta para el Sprint 1 y el Sprint Backlog
Observaciones:	Debe referenciar la Bibliografía consultada.

Ejemplo de Template de Minuta para Sprint Planning

Sprint Nro. <<X>>

Duración del Sprint en días: <<XX>>

Objetivo del Sprint: <<describa el objetivo del incremento de producto que se obtendrá al finalizar el sprint>>

Equipo Scrum:

<<Nombre del Miembro del Equipo>>

<<Nombre del Miembro del Equipo>>

<<Nombre del Miembro del Equipo>>

<<Nombre del Miembro del Equipo>>

<<Nombre del Miembro del Equipo>>

<<Nombre del Miembro del Equipo Automatizado/IA>>

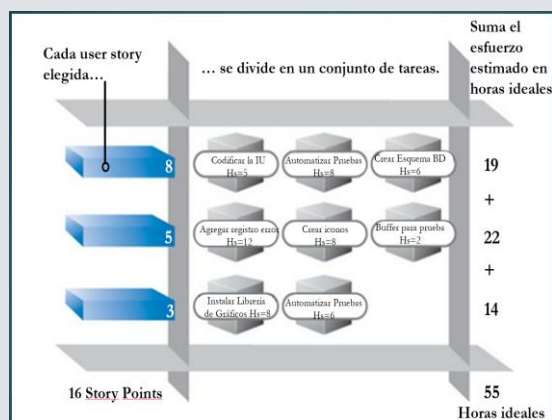
Capacidad del Equipo en Horas Ideales: <<XXX>>

Definición de Hecho para el Equipo

Definición de Hecho (DONE)

- ☐ Diseño revisado
- ☐ Código Completo
- ☐ Código refactorizado
- ☐ Código con formato estándar
- ☐ Código Comentado
- ☐ Código en el repositorio
- ☐ Código Inspeccionado
- ☐ Documentación de Usuario actualizada
- ☐ Probado
- ☐ Prueba de unidad hecha
- ☐ Prueba de integración hecha
- ☐ Prueba de Regresión hecha
- ☐ Plataforma probada
- ☐ Lenguaje probado
- ☐ Cero defectos conocidos
- ☐ Prueba de Aceptación realizada

Ejemplo de Sprint Backlog



Taxi Mobile: Sistema web mobile para seguimiento de taxis

Objetivo: Desarrollar un producto de software para Smartphones, donde los pasajeros de taxis puedan solicitar el taxi más cercano y saber su ubicación y demora en todo momento.

Se describen a continuación los principales roles de usuario:

Rol de Usuario	Descripción
 Pasajero	Persona que va a utilizar la aplicación con mucha frecuencia, para solicitar que le envíen un taxi al lugar donde está ubicado. Para él la simplicidad es importante. Está familiarizado con el uso de smartphones y puede instalar la aplicación. Su expectativa es que el taxi llegue a buscarlo lo más rápido posible.
 Taxista	Está familiarizado con el uso de smartphones y puede instalar la aplicación. Debe utilizar la aplicación mientras está trabajando, por eso necesita recibir solicitudes de viaje con notificaciones sonoras para enterarse de que tiene una solicitud de viaje. También debe poder utilizar la aplicación con manos libres. Necesita visualizar la posición del pasajero para poder llegar y necesita minimizar la cantidad de interacciones con el celular para informar que se dirige a buscar un pasajero o que ya lo encontró.
 Administrador de Central de Taxis	Debe poder trabajar con aplicaciones web y está familiarizado con el uso de Google maps. Necesita poder visualizar los taxis con su estado y poder visualizar la localización de los pasajeros para poder asistir a los Taxistas. La información debe estar disponible en tiempo real.

MVP (Producto Mínimo Viable)

Objetivo:

- Permitir que los clientes (pasajeros) puedan trasladarse al lugar que desean solicitando un taxi cercano.
- Permitir a los taxistas ver la ubicación del pasajero que solicita un viaje para llevarlo.
- Permitir a los taxistas manejar el estado del taxi.

Criterio:

Las user stories incluidas permiten validar la idea de negocio en el mercado, focalizando en la funcionalidad que pone en contacto a Pasajeros con Taxistas. Si bien los taxistas deben poder iniciar sesión para vincularlos con los datos de su vehículo, el análogo para el pasajero no es requisito para poder realizar un pedido. La funcionalidad relacionada con la gestión de taxis por parte de la central no aporta el valor significativo para la validación del mercado por lo que se dejará para futuras iteraciones.

Frases Verbales de las historias de usuario incluidas en el MVP:

Taxista

- Loguear taxista
- Ocupar taxi
- Liberar taxi
- Ver ubicación del pasajero

Pasajero

- Buscar taxis cercanos
- Pedir taxi
- Notificar a taxista solicitud de taxi

Como pasajero quiero loguearme para poder visualizar los taxis más cercanos. Criterios de Aceptación: - Datos para el login: nombre, apellido, teléfono celular (opcional). - Pueden tomarse los datos de Facebook o desde el mismo celular.	
Pruebas de Usuario ☐ Probar obtener datos desde Facebook con conexión a internet vía wifi. (pasa) ☐ Probar obtener datos desde Facebook con conexión a internet vía 3g. (pasa) ☐ Probar obtener datos desde Facebook sin conexión a internet. (falla) ☐ Probar solicitar los datos al pasajero. (pasa)	

Justificación de estimación:

- Complejidad:** user story simple ya que requiere un ingreso de datos mínimos y la interfaz con Facebook, para lo cual se utilizará una solución existente.
- Esfuerzo:** Se requiere esfuerzo mínimo vinculado a utilizar la solución existente para la interfaz con Facebook.
- Incertidumbre:** Mínimo, vinculado a la duda técnica respecto a cómo utilizar las APIs correspondientes para resolver la interfaz con Facebook.

Loguear taxista	2
Como taxista quiero loguearme para poder visualizar los pedidos de taxis. Criterios de Aceptación: Datos para el login: nombre, apellido, teléfono celular, dominio, número de móvil, central.	
Pruebas de Usuario ☐ Probar ingresar los datos completos cuando el taxista se encuentra asociado a una central. (pasa) ☐ Probar ingresar los datos cuando el taxista no se encuentra asociado a una central. (falla)	

Justificación de estimación:

- Complejidad:** User story simple ya que requiere un ingreso de datos mínimos y la interfaz con Facebook, para lo cual se utilizará una solución existente.
- Esfuerzo:** Se requiere esfuerzo mínimo vinculado a utilizar la solución existente para la interfaz con Facebook.
- Incertidumbre:** Mínimo, vinculado a la duda técnica respecto a cómo utilizar las APIs correspondientes para resolver la interfaz con Facebook. Esta duda será resuelta en la user story de logueo de pasajero.

Registrar central de taxis [user story canónica]	1
Como administrador de central quiero dar de alta la central para poder tomar viajes con Taxi-mobile. Criterio de Aceptación: El nombre de la central no debe repetirse.	
Pruebas de Usuario ☐ Probar registrar una central inexistente. (pasa) ☐ Probar registrar una central existente. (falla)	

Justificación de estimación:

- Complejidad:** User story simple ya que requiere un ingreso de datos mínimos y validaciones simples.
- Esfuerzo:** Se requiere esfuerzo mínimo vinculado a validaciones simples de unicidad.
- Incertidumbre:** Nula, no hay duda respecto a los datos y validaciones requeridos para registrar una central.

Se la selecciona como la **user story canónica** por ser de poca complejidad e incertidumbre nula, por lo que es factible de ser comparable con otras user stories.

Pedir taxi	5
Como pasajero quiero poder pedir un taxi seleccionando el más conveniente de un mapa para asegurarme de que el taxi está cerca.	

Universidad Tecnológica Nacional – Facultad Regional Córdoba Cátedra de Ingeniería y Calidad de Software – 2026 Curso: 4K1 - 4k2 - 4k3	
Criterios de Aceptación: - El celular debe contar con el sistema de geoposicionamiento online. - El pasajero se debe visualizar gráficamente en un mapa.	
Pruebas de Usuario ☐ Probar seleccionar el taxi de entre un conjunto de taxis visualizados (pasa) ☐ Probar seleccionar un taxi cuando no existen taxis disponibles (realizando una selección en cualquier parte de la pantalla). (falla) ☐ Probar visualizar los taxis cuando no está activo el sistema de posicionamiento del celular(falla)	

Justificación de estimación:

- Complejidad:** User story compleja ya que requiere la vinculación con el sistema de geoposicionamiento online y la generación de un mapa.
- Esfuerzo:** Se requiere esfuerzo alto para resolver la generación del mapa y la ubicación del taxi a partir de la respuesta del sistema de geoposicionamiento. Se requiere esfuerzo de investigación y pruebas.
- Incertidumbre:** Media. Existen soluciones probadas para generar el mapa y establecer una interfaz con el sistema de geoposicionamiento, pero deben probarse y definirse la solución más adecuada.

Ocupar taxi	2
Como taxista quiero marcar que el taxi se encuentra ocupado para no recibir pedidos de servicio que no podrá atender.	
Criterios de Aceptación: Para ocupar el taxi, este debe estar libre o con un pedido pendiente	
Pruebas de Usuario ☐ Probar ocupar el taxi cuando existe un viaje pedido pendiente. (pasa) ☐ Probar ocupar el taxi cuando está libre. (pasa) ☐ Probar ocupar el taxi cuando ya está ocupado. (falla)	

Justificación de estimación:

- Complejidad:** User story simple ya que requiere un único dato y cambio de estado de una única entidad (el taxi).
- Esfuerzo:** Poco esfuerzo para el desarrollo y medio para realización de pruebas de taxis en distintos estados.
- Incertidumbre:** Nula, no hay duda respecto a cómo realizar el cambio de estado.

Liberar taxi	2
Como taxista quiero liberar el taxi cuando estaba ocupado para que esté disponible para un próximo pedido de viaje.	
Criterios de Aceptación: Para liberar el taxi, este debe estar en curso	
Pruebas de Usuario ☐ Probar liberar el taxi cuando está con un viaje en curso. (pasa) ☐ Probar liberar el taxi cuando está libre. (falla)	

Justificación de estimación:

- Complejidad:** User story simple ya que requiere un único dato y cambio de estado de una única entidad (el taxi).
- Esfuerzo:** Poco esfuerzo para el desarrollo y medio para realización de pruebas de taxis en distintos estados.
- Incertidumbre:** Nula, no hay duda respecto a cómo realizar el cambio de estado.

Marcar taxi como fuera de servicio	2
Como taxista quiero marcar que el taxi se encuentra fuera de servicio para no recibir pedidos de servicio que no podrá atender.	
Criterios de Aceptación:	

- El taxi queda en estado fuera de servicio.
- El taxi no puede pasar a fuera de servicio si está ocupado o con pedido de viaje pendiente.

Pruebas de Usuario

- ☐ Probar marcar el taxi como fuera de servicio cuando el taxi está libre. (pasa)
- ☐ Probar marcar el taxi como fuera de servicio cuando existe un viaje pedido pendiente. (falla)
- ☐ Probar marcar el taxi como fuera de servicio cuando ya está ocupado. (falla)

Justificación de estimación:

- **Complejidad:** user story simple ya que requiere un único dato y cambio de estado de una única entidad (el taxi).
- **Esfuerzo:** Poco esfuerzo para el desarrollo y medio para realización de pruebas de taxis en distintos estados.
- **Incertidumbre:** Nula, no hay duda respecto a cómo realizar el cambio de estado.

Notificar a taxista y a central pedido de taxi

3

Como pasajero quiero enviar una notificación al momento en que solicito un viaje para que el taxista me busque y la central esté enterada del pedido.

Criterios de Aceptación:

- La notificación debe llegar en forma simultánea al taxista y a la central.

Pruebas de Usuario

- ☐ Probar notificar al taxista y la central cuando ambos tienen conexión de internet. (pasa)
- ☐ Probar notificar al taxista y la central cuando ninguno tiene conexión de internet. (falla)

Justificación de estimación:

- **Complejidad:** user story de complejidad media ya que requiere resolver la forma en que se enviará un mensaje push.
- **Esfuerzo:** Esfuerzo de investigación y desarrollo para resolver el envío de mensaje push, y de preparación de ambiente y pruebas.
- **Incertidumbre:** Media, vinculada a la duda técnica respecto a cómo enviar el mensaje push, a partir de soluciones existentes.

Buscar taxis cercanos

3

Como pasajero quiero ver cuáles son los taxis más cercanos a mi ubicación para pedir el taxi que más me convenga.

Criterios de Aceptación (CA):

- Se muestra la ubicación del taxi y el tiempo estimado para llegar al lugar en el que se encuentra el pasajero.
- Se mostrarán hasta 5 taxis cercanos.

Pruebas de Usuario

- ☐ Probar visualizar taxis cuando existen al menos 5 taxis libres. (pasa)
- ☐ Probar visualizar taxis cuando no existe ningún taxi libre (falla)

Justificación de estimación:

- **Complejidad:** user story de complejidad media ya que requiere resolver la forma de obtener datos de ubicación y tiempo a partir del sistema de geoposicionamiento.
- **Esfuerzo:** Esfuerzo medio de investigación y desarrollo para resolver la obtención de datos de geoposicionamiento y tiempos, así como para la preparación del ambiente simulado para realizar pruebas. Se requiere gran número de combinaciones de pruebas.
- **Incertidumbre:** Media, vinculada a resolver la forma de obtener los datos de geoposicionamiento, y la simulación para realizar las pruebas.

Ver ubicación del pasajero

5

Como taxista quiero ver la ubicación del pasajero que ha solicitado un viaje para poder ir a buscarlo

Criterios de Aceptación (CA):

- El celular debe contar con el sistema de posicionamiento online.

- El pasajero se debe visualizar gráficamente en un mapa.

Pruebas de Usuario

- ☐ Probar visualizar ubicación del pasajero para un pedido de viaje pendiente. (pasa)
- ☐ Probar visualizar ubicación del pasajero para un pedido de viaje cuando no se ha solicitado ninguno. (falla)
- ☐ Probar visualizar ubicación del pasajero para un pedido de viaje cuando no está activo el sistema de ubicación del celular. (falla)

Justificación de estimación:

- Complejidad:** User story de complejidad alta ya que requiere resolver la forma de obtener datos de ubicación a partir del sistema de geoposicionamiento y la visualización en un mapa.
- Esfuerzo:** Esfuerzo medio de investigación y desarrollo para resolver la obtención de datos de geoposicionamiento, así como para la preparación del ambiente simulado para realizar pruebas. Se requiere gran número de combinaciones de pruebas.
- Incertidumbre:** Media, vinculada a resolver la forma de obtener los datos de geoposicionamiento, la simulación para realizar las pruebas, así como la generación del mapa.

Ver mapa de taxis	5
Como administrador de la central quiero ver la ubicación de todos los taxis de la central y si tienen viajes en curso para saber la disponibilidad actual	
Criterios de Aceptación: - Se muestran taxis en todos los estados. - Los taxis que se mostrarán deben tener habilitado el sistema de ubicación.	
Pruebas de Usuario ☐ Probar visualizar un conjunto de taxis libres , fuera de servicio y ocupados (pasa) ☐ Probar visualizar el mapa cuando ningún taxi de la central está conectado al sistema de ubicación (falla)	

Justificación de estimación:

- Complejidad:** user story de complejidad alta ya que requiere resolver la forma de obtener datos de ubicación a partir del sistema de geoposicionamiento y la generación del mapa con todos los taxis.
- Esfuerzo:** Esfuerzo medio de investigación y desarrollo para resolver la obtención de datos de geoposicionamiento, así como para la preparación del ambiente simulado para realizar pruebas. Se requiere gran número de combinaciones de pruebas.
- Incertidumbre:** Media, vinculada a resolver la forma de obtener los datos de geoposicionamiento, la simulación para realizar las pruebas y la generación del mapa.